

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 16 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMMA)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127 #include <cuda/ptx>
128
129 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
130 #error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
131 #endif
132
133 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
134 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
135 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
136
137 static constexpr int kWarpSize = 32;
138
139 // =====
140 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
141 // =====
142
143 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
144 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
145 // 复杂度  $O(\log N)$ ,  $N=32$  时仅需 5 步
146 // 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
147 // kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
148 // 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时, 只有同 segment 的 lane 参与通信
149 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
150 //
151 // 初始: 每个 lane 持有自己的值 v0..v7
152 // lane:  0    1    2    3    4    5    6    7
153 // val:  v0   v1   v2   v3   v4   v5   v6   v7
154 //
155 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
156 //      ┌───────────┐
157 // 对:  (0,4) (1,5) (2,6) (3,7)
158 //
159 // lane:  0    1    2    3    4    5    6    7
160 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
161 //
162 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
163 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
164 // 对:  (0,2) (1,3) (4,6) (5,7)
165 //      ─── 前4个一组 ───      ─── 后4个一组 ───
166 //
167 // lane:  0    1    2    3    4    5    6    7 (每lane持有前一轮2个值的和再加本轮配对)
168 // val:  $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$ 
169 //      = v0+v2+v4+v6 ... (逐步归约)
170 //
171 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
172 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
173 // 对:  (0,1) (2,3) (4,5) (6,7)
174 //
175 // lane:  0    1    2    3    4    5    6    7
176 // val:  $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$   $\Sigma_{all}$  ← 所有 lane 拥有全归约结果!
177 //
178 // mask=0: 循环终止, 归约完成。
179 //
180 // 关键性质:
181 // - XOR 配对是对称的: lane i 的配对对象是 lane  $i^{mask}$ , 而  $(i^{mask})^{mask} = i$ 
182 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
183 // - 每轮信息传递距离减半:  $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$  (距离减半, 信息翻倍)
184 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
185 // - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width:
186 //   当 kWarpWidth=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
187 template <const int kWarpWidth = kWarpSize, typename T = float>
188 __device__ __forceinline__ T warp_reduce_sum(T val) {

```

```

189 #pragma unroll
190 for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
191     val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
192 }
193 return val;
194 }
195
196 // Warp Reduce Max — generic
197 template <const int kWarpWidth = kWarpSize, typename T = float>
198 __device__ __forceinline__ T warp_reduce_max(T val) {
199 #pragma unroll
200     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
201         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
202     }
203     return val;
204 }
205
206 // =====
207 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
208 // =====
209
210 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
211 // 两级归约流程:
212 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
213 // 2. warp leader (lane=0) 写入 shared memory
214 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
215 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
216 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
217 template <const int kNumThreads = 256>
218 __device__ float block_reduce_sum(float val) {
219     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
220     int warp = threadIdx.x / kWarpSize;
221     int lane = threadIdx.x % kWarpSize;
222     __shared__ float shared[kNumWarps];
223
224     float value = warp_reduce_sum<kWarpSize>(val);
225     if (lane == 0)
226         shared[warp] = value;
227     __syncthreads();
228     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
229     value = warp_reduce_sum<kNumWarps>(value);
230     // 关键: broadcast 结果到所有线程, 后续用 result
231     // 做除法等操作时所有线程都能拿到
232     value = __shfl_sync(0xffffffff, value, 0, 32);
233     return value;
234 }
235
236 // Block Reduce Max — FP32 (增强版, 带 broadcast)
237 template <const int kNumThreads = 256>
238 __device__ float block_reduce_max(float val) {
239     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
240     int warp = threadIdx.x / kWarpSize;
241     int lane = threadIdx.x % kWarpSize;
242     __shared__ float shared[kNumWarps];
243
244     float value = warp_reduce_max<kWarpSize>(val);
245     if (lane == 0)
246         shared[warp] = value;
247     __syncthreads();
248     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
249     value = warp_reduce_max<kNumWarps>(value);
250     value = __shfl_sync(0xffffffff, value, 0, 32);
251     return value;

```

```

252 }
253
254 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
255 // 多 block 各自做 warp→smem→warp0 reduce, 然后 atomicAdd 到全局 y
256 // 跨 block 求和的常见模式, 适合 N 较大时使用;
257 // Grid: ((N + 255) / 256, 1, 1)
258 // Block: (256, 1, 1)
259 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
260 template <const int kNumThreads = 256>
261 __global__ void block_reduce_all(float *a, float *y, int N) {
262     int tid = threadIdx.x;
263     int idx = blockIdx.x * kNumThreads + tid;
264     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
265     __shared__ float shared[kNumWarps];
266
267     float val = (idx < N) ? a[idx] : 0.0f;
268     int warp = tid / kWarpSize;
269     int lane = tid % kWarpSize;
270
271     val = warp_reduce_sum<kWarpSize>(val);
272     if (lane == 0)
273         shared[warp] = val;
274     __syncthreads();
275
276     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
277     if (warp == 0)
278         val = warp_reduce_sum<kNumWarps>(val);
279     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
280         atomicAdd(y, val);
281 }
282
283 // ---- Dot Product: y = sum(a[i] * b[i]) ----
284 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
285 // Grid: ((N + 255) / 256, 1, 1)
286 // Block: (256, 1, 1)
287 // source: LeetCUDA/kernels/dot-product/dot_product.cu
288 template <const int kNumThreads = 256>
289 __global__ void dot(float *a, float *b, float *y, int N) {
290     int tid = threadIdx.x;
291     int idx = blockIdx.x * kNumThreads + tid;
292     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
293     __shared__ float shared[kNumWarps];
294
295     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
296     int warp = tid / kWarpSize;
297     int lane = tid % kWarpSize;
298
299     prod = warp_reduce_sum<kWarpSize>(prod);
300     if (lane == 0)
301         shared[warp] = prod;
302     __syncthreads();
303
304     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
305     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
306         prod = warp_reduce_sum<kNumWarps>(prod);
307     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
308         atomicAdd(y, prod);
309 }
310
311 // Dot Product + float4
312 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
313 // Block: (64, 1, 1), 256/4=64
314 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景

```

```

315 // source: LeetCUDA/kernels/dot-product/dot_product.cu
316 template <const int kNumThreads = 256 / 4>
317 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
318     int tid = threadIdx.x;
319     int idx = (blockIdx.x * kNumThreads + tid) * 4;
320     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
321     __shared__ float shared[kNumWarps];
322
323     float4 reg_a = FLOAT4(a[idx]);
324     float4 reg_b = FLOAT4(b[idx]);
325     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
326                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
327                          : 0.0f;
328     int warp = tid / kWarpSize;
329     int lane = tid % kWarpSize;
330
331     prod = warp_reduce_sum<kWarpSize>(prod);
332     if (lane == 0)
333         shared[warp] = prod;
334     __syncthreads();
335
336     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
337     if (warp == 0)
338         prod = warp_reduce_sum<kNumWarps>(prod);
339     if (tid == 0)
340         atomicAdd(y, prod);
341 }
342
343
344 // =====
345 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
346 // =====
347 // 面试要点:
348 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
349 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
350 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
351
352 // ---- ReLU: y = max(0, x) ----
353 // Grid: ((N + 255) / 256, 1, 1)
354 // Block: (256, 1, 1)
355 // source: LeetCUDA/kernels/relu/relu.cu
356 __global__ void relu(float *x, float *y, int N) {
357     int idx = blockIdx.x * blockDim.x + threadIdx.x;
358     if (idx < N)
359         y[idx] = fmaxf(0.0f, x[idx]);
360 }
361
362 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
363 // block(64)x4(float4)=256 元素/block, 与基础版吞吐相同
364 // Grid: ((N + 255) / 256, 1, 1)
365 // Block: (64, 1, 1)
366 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
367 // source: LeetCUDA/kernels/relu/relu.cu
368 __global__ void relu_vec4(float *x, float *y, int N) {
369     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
370     if (idx < N) {
371         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
372         float4 reg_y;
373         reg_y.x = fmaxf(0.0f, reg_x.x);
374         reg_y.y = fmaxf(0.0f, reg_x.y);
375         reg_y.z = fmaxf(0.0f, reg_x.z);
376         reg_y.w = fmaxf(0.0f, reg_x.w);
377         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store

```

```

378     }
379 }
380
381 // ---- Elementwise Add: c = a + b ----
382 // Grid: ((N + 255) / 256, 1, 1)
383 // Block: (256, 1, 1)
384 // source: LeetCUDA/kernels/elementwise/elementwise.cu
385 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
386     int idx = blockIdx.x * blockDim.x + threadIdx.x;
387     if (idx < N)
388         c[idx] = a[idx] + b[idx];
389 }
390
391 // Elementwise Add + float4 向量化
392 // Grid: ((N + 255) / 256, 1, 1)
393 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
394 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
395 // source: LeetCUDA/kernels/elementwise/elementwise.cu
396 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
397     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
398     if ((idx + 3) < N) {
399         float4 reg_a = FLOAT4(a[idx]);
400         float4 reg_b = FLOAT4(b[idx]);
401         float4 reg_c;
402         reg_c.x = reg_a.x + reg_b.x;
403         reg_c.y = reg_a.y + reg_b.y;
404         reg_c.z = reg_a.z + reg_b.z;
405         reg_c.w = reg_a.w + reg_b.w;
406         FLOAT4(c[idx]) = reg_c;
407     } else if (idx < N) {
408         for (int i = 0; (idx + i) < N; i++) {
409             c[idx + i] = a[idx + i] + b[idx + i];
410         }
411     }
412 }
413
414 // ---- Histogram: y[a[i]]++ ----
415 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
416 // Grid: ((N + 255) / 256, 1, 1)
417 // Block: (256, 1, 1)
418 // source: LeetCUDA/kernels/histogram/histogram.cu
419 __global__ void histogram(int *a, int *y, int N) {
420     int idx = blockIdx.x * blockDim.x + threadIdx.x;
421     if (idx < N)
422         atomicAdd(&(y[a[idx]]), 1);
423 }
424
425 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
426 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
427 //
428 // 面试要点:
429 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
430 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
431 //   - 数学公式 (5 步推导):
432 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
433 //         L_max = max(LSE_1, LSE_2)
434 //     Step 2 — 还原指数权重 (un-normalized):
435 //         w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
436 //     Step 3 — 归一化为混合比例:
437 //         alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
438 //         满足 alpha + beta = 1
439 //     Step 4 — 逐元素加权合并:
440 //         O = alpha * O_1 + beta * O_2

```



```

441 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
442 //   lse[head_idx][token_idx])
443 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
444 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
445 // - inf LSE → -inf: 空 attention 段 (causal mask 等导致全部 score
446 //   为 -inf) 的 LSE 可能为 +inf, 替换后  $\exp(-\text{inf} - L_{\text{max}}) = 0$ ,
447 //   该段权重退化为 0
448 // - 128-bit 向量化: uint4 = 4 × float, 每线程处理 4 个输出元素,
449 //   pack load → FMA → pack store 一条流水线
450 // - AI ≈ 3 FLOP / 20 bytes ≈ 0.15 FLOPS/Byte → severely memory-bound
451 //
452 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
453 //   kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
454 //   threads_per_head = head_size / 4 = 2
455 //   total_threads = num_tokens * num_heads * threads_per_head = 8
456 //
457 //   global_idx:      0      1      2      3      4      5      6      7
458 //   token_head_idx:  0      0      1      1      2      2      3      3
459 //       (第几个 (token,head) 对, 行优先)
460 //   pack_idx:        0      1      0      1      0      1      0      1
461 //       (该 (token,head) 对内第几个 pack)
462 //   token_idx:       0      0      0      0      1      1      1      1
463 //       (token_head_idx / num_heads, token 变化慢)
464 //   head_idx:        0      0      1      1      0      0      1      1
465 //       (token_head_idx % num_heads, head 变化快)
466 //   pack_offset:     0      4      0      4      0      4      0      4
467 //       (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
468 //   head_offset:     0      0      8      8      16     16     24     24
469 //       (token_idx * num_heads * head_size + head_idx * head_size,
470 //       该 (token,head) 对在 output 展平数组中的起始偏移)
471 //
472 // Grid: ((total_threads + 127) / 128, 1, 1)
473 // Block: (128, 1, 1)
474 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
475 __global__ void merge_attn_states(
476     float *output,           // [num_tokens, num_heads, head_size]
477     const float *prefix_output, // [num_tokens, num_heads, head_size]
478     const float *prefix_lse,   // [num_heads, num_tokens]
479     const float *suffix_output, // [num_tokens, num_heads, head_size]
480     const float *suffix_lse,   // [num_heads, num_tokens]
481     int num_tokens, int num_heads, int head_size) {
482
483     constexpr int kNumThreads = 128;
484     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
485     using pack_t = uint4;
486
487     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
488     const int threads_per_head = head_size / kPackSize;
489     // 实际有效总线线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
490     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
491     const int total_threads = num_tokens * num_heads * threads_per_head;
492
493     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
494     if (global_idx >= total_threads)
495         return;
496
497     // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
498     // token_head_idx: 第几个 (token, head) 对, 行优先展平
499     const int token_head_idx = global_idx / threads_per_head;
500     // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
501     const int pack_idx = global_idx % threads_per_head;
502
503     // token_head_idx 分解为 (token_idx, head_idx)

```

```

504 const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
505 const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
506
507 // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
508 const int pack_offset = pack_idx * kPackSize;
509 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
510 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
511
512 // 定位到当前 (token, head) 的输出段起始
513 const float *prefix_head = prefix_output + head_offset;
514 const float *suffix_head = suffix_output + head_offset;
515 float *output_head = output + head_offset;
516
517 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
518 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
519 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
520
521 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
522 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
523 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
524
525 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
526 const float max_lse = fmaxf(p_lse, s_lse);
527 p_lse -= max_lse;
528 s_lse -= max_lse;
529
530 // Step 2-3: 指数还原 → 混合比例 alpha, beta
531 const float p_se = expf(p_lse);
532 const float s_se = expf(s_lse);
533 const float out_se = p_se + s_se;
534 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
535 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
536
537 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
538 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
539 if (pack_offset < head_size) {
540     pack_t p_pack =
541         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
542     pack_t s_pack =
543         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
544     pack_t o_pack;
545
546 #pragma unroll
547     for (int i = 0; i < kPackSize; ++i) {
548         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
549         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
550         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
551         reinterpret_cast<float *>(&o_pack)[i] = o_v;
552     }
553
554     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
555 }
556 }
557
558 // =====
559 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
560 // =====
561 // 面试要点:
562 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
563 //     online (1-pass, 增量更新)
564 //   - Online Softmax 是 FlashAttention 的数学基础
565 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
566 //     + variance)

```

```

567 // - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
568
569 // =====
570 // Phase 3a: Softmax — 三级递进 (面试核心考点)
571 // =====
572 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
573 // 回答线索: naive(溢出) → safe → online
574
575 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
576 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
577 // 问题: x 值很大时 exp(x) 溢出为 inf
578 template <const int kNumThreads = 256>
579 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
580 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
581 // source: LeetCUDA/kernels/softmax/softmax.cu
582 __global__ void softmax_per_token(float *x, float *y, int N) {
583     const int tid = threadIdx.x;
584     const int idx = blockIdx.x * blockDim.x + tid;
585
586     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
587     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
588     if (idx < N)
589         y[idx] = exp_val / exp_sum;
590 }
591
592 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
593 // 面试重点: 为什么 Softmax 需要 Safe?
594 // - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
595 // - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
596 // - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
597 // - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
598 // Grid: (S, 1, 1)
599 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
600 // source: LeetCUDA/kernels/softmax/softmax.cu
601 template <const int kNumThreads = 256>
602 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
603     const int tid = threadIdx.x;
604     const int idx = blockIdx.x * blockDim.x + tid;
605
606     // Pass 1: block reduce max — 找最大值
607     float val = (idx < N) ? x[idx] : -FLT_MAX;
608     float max_val = block_reduce_max<kNumThreads>(val);
609
610     // Pass 2: exp(x - max) → block reduce sum
611     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
612     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
613
614     if (idx < N)
615         y[idx] = exp_val / exp_sum;
616 }
617
618 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
619 // 面试重点:
620 // 两种使用场景 (公式不同, 但结果等价):
621 // 1) 单元素增量更新 (online update, 处理新元素 x_i):
622 //     m_new = max(m_old, x_i)
623 //     d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
624 // 2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
625 //     m = max(m1, m2) safe softmax用的max值
626 //     d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
627 //     当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
628 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
629 // Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.

```

```

630 //
631 // 不同于单元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
632 // warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
633 // (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
634 // (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
635 //
636 // 合并公式 (对称, 不依赖"新/旧"概念) :
637 // m = max(m1, m2)
638 // d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
639 //
640 // 该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
641 struct __align__(8) MD {
642     float m; // running max
643     float d; // running denominator (sum of exp(x - max))
644 };
645
646 template <const int kWarpWidth = kWarpSize>
647 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
648     #pragma unroll
649     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
650         MD md2;
651         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
652         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);
653
654         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
655         MD s_m = (md1.m > md2.m) ? md2 : md1;
656
657         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
658         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
659         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
660         md1.m = b_m.m;
661     }
662     return md1;
663 }
664
665 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
666 // Grid: (S, 1, 1)
667 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
668 // source: LeetCUDA/kernels/softmax/softmax.cu
669 template <const int kNumThreads = 256>
670 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
671     int tid = threadIdx.x;
672     int idx = blockIdx.x * kNumThreads + threadIdx.x;
673     const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
674     __shared__ MD shared[kNumWarps];
675
676     float val = (idx < N) ? x[idx] : -FLT_MAX;
677     int warp = tid / kWarpSize;
678     int lane = tid % kWarpSize;
679
680     // 初始化: 每个线程持有一个 (max, denom) 对
681     MD md;
682     md.m = idx < N ? val : -FLT_MAX;
683     md.d = idx < N ? 1.0f : 0.0f;
684
685     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d
686     md = warp_reduce_md<kWarpSize>(md);
687
688     if (lane == 0)
689         shared[warp] = md;
690     __syncthreads();
691
692     // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)

```

```

693 md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
694 md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果
695
696 // 用全局 max 和 denom 做最终 softmax
697 float d_inv = __fdivdef(1.0f, md.d);
698 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
699 if (idx < N) {
700     y[idx] = __expf(val - md.m) * d_inv;
701 }
702 }
703
704 // =====
705 // Phase 3b: RMS Normalization (1-pass reduce)
706 // =====
707 // 面试要点:
708 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
709 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
710 //   - Llama 系列使用 RMS Norm
711 //   - grid(N, K/K), block(K): 一行一个 block
712 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
713 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
714 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
715 template <const int kNumThreads = 128>
716 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
717     int tid = threadIdx.x;
718     int idx = blockIdx.x * blockDim.x + threadIdx.x;
719     const float epsilon = 1e-5f;
720
721     __shared__ float s_variance;
722     float value = (idx < N * K) ? x[idx] : 0.0f;
723     float variance = value * value;
724     variance = block_reduce_sum<kNumThreads>(variance);
725     if (tid == 0)
726         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
727     __syncthreads();
728     if (idx < N * K)
729         y[idx] = (value * s_variance) * g;
730 }
731
732 // RMS Norm + float4
733 // Grid: (N, 1, 1)
734 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
735 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
736 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
737 template <const int kNumThreads = 128 / 4>
738 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
739     int tid = threadIdx.x;
740     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
741     const float epsilon = 1e-5f;
742
743     __shared__ float s_variance;
744     float4 reg_x = FLOAT4(x[idx]);
745     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
746                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
747                                     : 0.0f;
748     variance = block_reduce_sum<kNumThreads>(variance);
749     if (tid == 0)
750         s_variance = rsqrtf(variance / (float)K + epsilon);
751     __syncthreads();
752     float4 reg_y;
753     reg_y.x = reg_x.x * s_variance * g;
754     reg_y.y = reg_x.y * s_variance * g;
755     reg_y.z = reg_x.z * s_variance * g;

```

```

756 reg_y.w = reg_x.w * s_variance * g;
757 if (idx < N * K)
758     FLOAT4(y[idx]) = reg_y;
759 }
760
761 // =====
762 // Phase 3c: Layer Normalization (2-pass reduce)
763 // =====
764 // 面试要点:
765 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
766 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
767 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
768 // Grid: (N, 1, 1), 一行一个 block
769 // Block: (128, 1, 1), kNumThreads=K=128
770 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
771 template <const int kNumThreads = 128>
772 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
773     int tid = threadIdx.x;
774     int idx = blockIdx.x * blockDim.x + threadIdx.x;
775     const float epsilon = 1e-5f;
776
777     __shared__ float s_mean;
778     __shared__ float s_variance;
779     float value = (idx < N * K) ? x[idx] : 0.0f;
780
781     // Pass 1: compute mean
782     float sum = block_reduce_sum<kNumThreads>(value);
783     if (tid == 0)
784         s_mean = sum / (float)K;
785     __syncthreads(); // 必须等待 s_mean 对所有线程可见
786
787     // Pass 2: compute variance = (x - mean)2
788     float variance = (value - s_mean) * (value - s_mean);
789     variance = block_reduce_sum<kNumThreads>(variance);
790     if (tid == 0)
791         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
792     __syncthreads(); // 必须等待 s_variance 对所有线程可见
793
794     if (idx < N * K)
795         y[idx] = ((value - s_mean) * s_variance) * g + b;
796 }
797
798 // Layer Norm + float4
799 // Grid: (N, 1, 1)
800 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
801 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
802 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
803 template <const int kNumThreads = 128 / 4>
804 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
805     int tid = threadIdx.x;
806     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
807     const float epsilon = 1e-5f;
808
809     __shared__ float s_mean;
810     __shared__ float s_variance;
811     float4 reg_x = FLOAT4(x[idx]);
812     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
813
814     float sum = block_reduce_sum<kNumThreads>(value);
815     if (tid == 0)
816         s_mean = sum / (float)K;
817     __syncthreads();
818

```

```

819 float4 reg_x_hat;
820 reg_x_hat.x = reg_x.x - s_mean;
821 reg_x_hat.y = reg_x.y - s_mean;
822 reg_x_hat.z = reg_x.z - s_mean;
823 reg_x_hat.w = reg_x.w - s_mean;
824 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
825                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
826 variance = block_reduce_sum<kNumThreads>(variance);
827 if (tid == 0)
828     s_variance = rsqrtf(variance / (float)K + epsilon);
829 __syncthreads();
830
831 float4 reg_y;
832 reg_y.x = reg_x_hat.x * s_variance * g + b;
833 reg_y.y = reg_x_hat.y * s_variance * g + b;
834 reg_y.z = reg_x_hat.z * s_variance * g + b;
835 reg_y.w = reg_x_hat.w * s_variance * g + b;
836 if (idx < N * K)
837     FLOAT4(y[idx]) = reg_y;
838 }
839
840 // =====
841 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
842 // =====
843 // 面试要点:
844 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
845 //     [x1'] [cos(θ) -sin(θ)] [x1]
846 //     [x2'] [sin(θ)  cos(θ)] [x2]
847 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
848 //   - token_pos = idx / N: token 在序列中的位置
849 //   - token_idx = idx % N: token 内的维度对索引
850 //   - 输入 [seq_len, hidden_size], 输出同形状
851
852 // Grid: ((seq_len * N + 255) / 256, 1, 1)
853 // Block: (256, 1, 1)
854 // source: LeetCUDA/kernels/rope/rope.cu
855 __global__ void rope(float *x, float *out, int seq_len, int N) {
856     int idx = blockIdx.x * blockDim.x + threadIdx.x;
857     float x1 = x[idx * 2];
858     float x2 = x[idx * 2 + 1];
859     int token_pos = idx / N; // 序列位置
860     int token_idx = idx % N; // 维度对索引
861
862     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
863     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
864     float sin_v = sinf(token_pos * exp_v);
865     float cos_v = cosf(token_pos * exp_v);
866
867     // 2D 旋转
868     float out1 = x1 * cos_v - x2 * sin_v;
869     float out2 = x1 * sin_v + x2 * cos_v;
870     out[idx * 2] = out1;
871     out[idx * 2 + 1] = out2;
872 }
873
874 // =====
875 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
876 // =====
877 // 面试要点 (Bank Conflict 专题):
878 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
879 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
880 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
881 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)

```



```

882 //
883 // 转置的四步演进:
884 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
885 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
886 //   BCF:   smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
887 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
888 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
889
890 // 每个线程处理 1 个元素, block(16,16)
891 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
892 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
893 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
894 // Block: (16, 16, 1)
895 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
896 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
897     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
898     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
899     if (r < row && c < col) {
900         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
901         y[c * row + r] = x[r * col + c];
902     }
903 }
904
905 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
906 // Block: (16, 16, 1)
907 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
908 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
909 __global__ void mat_transpose_padded(
910     float *x, float *y, const int row, const int col) {
911     const int tx = threadIdx.x;
912     const int ty = threadIdx.y;
913
914     constexpr int TILE = 16;
915     constexpr int PAD = 1;
916     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
917     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
918
919     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
920     const int x_c = blockIdx.x * TILE + tx;
921     const int x_r = blockIdx.y * TILE + ty;
922
923     if (x_r * 4 < row && x_c < col) {
924         // Step 1: 4 rows × 1 col → smem (row-major);
925 #pragma unroll
926         for (int i = 0; i < 4; ++i) {
927             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
928         }
929         __syncthreads();
930
931         // Step 2: Transposed read from smem
932         float4 reg_trans;
933         reg_trans.x = tile[tx * 4 + 0][ty];
934         reg_trans.y = tile[tx * 4 + 1][ty];
935         reg_trans.z = tile[tx * 4 + 2][ty];
936         reg_trans.w = tile[tx * 4 + 3][ty];
937
938         // y 空间坐标: y_r = x 的列, y_c = x 的行
939         const int y_r = blockIdx.x * TILE + ty; // = x col
940         const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
941
942         // Coalesced write: y[y_r][y_c .. y_c+3]
943         FLOAT4(y[y_r * row + y_c]) = reg_trans;
944     }

```



```

945 }
946
947 // =====
948 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
949 // =====
950 // 面试要点:
951 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
952 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
953 //   - 不同 K 值对应不同分块策略:
954 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
955 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
956 //       K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
957 //
958 // a: M×K, x: K×1, y: M×1, 计算:  $y = a * x$ ; N = 1
959
960 // ---- SGEMV K32: 基础 warp-per-row ----
961 // 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
962 // grid(M/4), 每个 warp 负责一行
963 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
964 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
965 // Block: (32, 4, 1), 每行 1 warp
966 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
967 // source: LeetCUDA/kernels/sgemv/sgemv.cu
968 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
969     int tx = threadIdx.x; // 0~31
970     int ty = threadIdx.y; // 0~3
971     int lane = tx % kWarpSize; // 0~31
972     int m = blockIdx.x * blockDim.y + ty; // 全局行号
973     if (m < M) {
974         float sum = 0.0f;
975         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
976         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
977         const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
978 #pragma unroll
979         for (int w = 0; w < NUM_ITERS; ++w) {
980             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
981             int k = w * kWarpSize + lane;
982             sum += a[m * K + k] * x[k];
983         }
984         sum = warp_reduce_sum<kWarpSize>(sum);
985         // 每个 warp 处理一行, lane 0 写回结果
986         if (lane == 0)
987             y[m] = sum;
988     }
989 }
990
991 // ---- SGEMV K128: float4 向量化 ----
992 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
993 // Grid: ((M + 3) / 4, 1, 1)
994 // Block: (32, 4, 1)
995 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
996 // source: LeetCUDA/kernels/sgemv/sgemv.cu
997 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
998     int tx = threadIdx.x; // 0~31
999     int ty = threadIdx.y; // 0~3
1000    int lane = tx % kWarpSize; // 0~31
1001    int m = blockDim.y * blockIdx.x + ty;
1002
1003    if (m < M) {
1004        float sum = 0.0f;
1005        // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1006        const int NUM_ITERS = (((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
1007 #pragma unroll

```

```

1008     for (int w = 0; w < NUM_ITERS; ++w) {
1009         int k = (w * kWarpSize + lane) * 4;
1010         float4 reg_x = FLOAT4(x[k]);
1011         float4 reg_a = FLOAT4(a[m * K + k]);
1012         sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1013             reg_a.w * reg_x.w);
1014     }
1015     sum = warp_reduce_sum<kWarpSize>(sum);
1016     if (lane == 0)
1017         y[m] = sum;
1018 }
1019 }
1020
1021 // ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
1022 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1023 // kRowPerWarp=2, kNumLanePerRow=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1024 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1025 // Block: (32, 4, 1)
1026 // 注意: 这一版是面向 K=16 的专用写法; kRowPerWarp=2 时一个 warp 同时处理 2 行
1027 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1028 template <const int kRowPerWarp = 2>
1029 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1030     constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
1031     int tx = threadIdx.x;
1032     int ty = threadIdx.y;
1033     int lane = tx % kWarpSize;
1034     int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
1035     int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
1036     if (m < M) {
1037         float sum = A[m * K + k] * x[k];
1038         // 按照kNumLanePerRow=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1039         sum = warp_reduce_sum<kNumLanePerRow>(sum);
1040         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1041         if (k == 0)
1042             y[m] = sum;
1043     }
1044 }
1045
1046 // =====
1047 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1048 // =====
1049 // 面试要点 (GEMM 优化五层金字塔):
1050 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1051 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1052 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1053 //   load/store 指令数 Level 4 — Tensor Core (MMA
1054 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1055 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1056 //
1057 // 计算密度递进:
1058 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1059 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1060 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1061
1062 // =====
1063 // Phase 7a: SGEMM (非 Tensor Core 路径)
1064 // =====
1065
1066 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
1067 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1068 // C = A x B, C[M, N] = A[M, K] x B[K, N]
1069 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1070 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)

```

```

1071 // Block: (32, 32, 1), 1024 线程
1072 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1073 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1074     constexpr int BM = 32; // vec 版: 32x4 = 128
1075     constexpr int BN = 32; // vec 版: 32x4 = 128
1076     constexpr int BK = 32;
1077     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1078
1079     int bx = blockIdx.x;
1080     int by = blockIdx.y;
1081     int tx = threadIdx.x;
1082     int tid = threadIdx.y * blockDim.x + tx;
1083
1084     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1085     // 技巧: 一般来说 “/” 表示线程不是连续排布的, “%” 表示线程是连续排布的
1086     // 因此, 在需要考虑连续访问的维度使用 “%”, 比如, 连续的线程访问列方向连续的元素
1087     // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
1088     // 线程不连续访问 M 维度 → 用 /;
1089     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1090     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1091     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1092     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1093     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1094     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1095
1096     float sum = 0.f; // 遍历完整的K, slice K;
1097     // 这里不用pragma unroll, 因为K不是编译器常量, 编译器无法展开循环
1098     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1099         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1100         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1101         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1102         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1103         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1104         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1105         __syncthreads(); // 确保整个 smem tile 加载完毕
1106
1107     #pragma unroll
1108         for (int k = 0; k < BK; ++k) {
1109             int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1110             int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1111             sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1112         }
1113         __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1114     }
1115     int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1116     int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1117     int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1118     c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1119 }
1120
1121 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1122 // 在 Level 1 基础上引入两层优化:
1123 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1124 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1125 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1126 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
1127 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1128 // 1024 x 16 = 16384 = 128 x 128 ✓
1129 //
1130 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1131 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1132 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1133 //

```

```

1134 // 加载映射 (每线程 4 个元素, float4) :
1135 //   A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
1136 //   B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
1137 //   row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1138 //
1139 // △ Bank Conflict 提示 (面试加分点) :
1140 //   s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1141 //   → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1142 //   s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1143 //
1144 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1145 // Block: (32, 32, 1), 1024 线程
1146 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1147 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1148 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1149     constexpr int BM = 128;
1150     constexpr int BN = 128;
1151     constexpr int BK = 32;
1152     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1153     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1154
1155     int bx = blockIdx.x;
1156     int by = blockIdx.y;
1157     int tx = threadIdx.x;
1158     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1159
1160     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1161     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1162     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1163     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1164     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1165     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1166
1167     int load_gmem_a_m = by * BM + load_smem_a_m;
1168     int load_gmem_b_n = bx * BN + load_smem_b_n;
1169
1170     // 4×4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1171     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1172     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1173     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1174     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1175     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1176
1177     float sum[4][4] = {0.f};
1178     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1179         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1180         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1181         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1182         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1183         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1184         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1185         __syncthreads();
1186
1187     #pragma unroll
1188     for (int k = 0; k < BK; ++k) {
1189         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4×4=16 次 FMA
1190         float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1191                             s_a[comp_smem_a_m_base + 1][k],
1192                             s_a[comp_smem_a_m_base + 2][k],
1193                             s_a[comp_smem_a_m_base + 3][k]};
1194         float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1195                             s_b[k][comp_smem_b_n_base + 1],
1196                             s_b[k][comp_smem_b_n_base + 2],

```

```

1197         s_b[k][comp_smem_b_n_base + 3]};
1198 #pragma unroll
1199     for (int i = 0; i < 4; ++i) {
1200 #pragma unroll
1201         for (int j = 0; j < 4; ++j) {
1202             sum[i][j] += a_vals[i] * b_vals[j];
1203         }
1204     }
1205 }
1206 __syncthreads();
1207 }
1208
1209 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1210 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1211 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1212 #pragma unroll
1213 for (int i = 0; i < 4; ++i) {
1214     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1215     float4 reg_c;
1216     reg_c.x = sum[i][0];
1217     reg_c.y = sum[i][1];
1218     reg_c.z = sum[i][2];
1219     reg_c.w = sum[i][3];
1220     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1221 }
1222 }
1223
1224 // =====
1225 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
1226 // =====
1227 // 面试要点:
1228 // - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1229 // - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1230 // - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1231 //   寄存器)
1232 // - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1233 // - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1234 //
1235 // ★ TN 布局详解 (面试高频考点):
1236 // TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1237 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1238 //   N = Normal (列优先, BLAS 原生格式)
1239 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1240 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1241 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1242 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1243 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1244 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1245 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1246 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1247 //
1248 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1249 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1250 //   N = col-major (列优先, BLAS native = normal)
1251 //
1252 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
1253 //   - A: row-major [M, K], B: row-major [K, N]
1254 //   - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1255 //   - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1256 //
1257 // TN 布局: C[MxN] = A[MxK] × B[KxN], B 以 B^T=[NxK] row-major 存储 (A 行优先, B 列优先)
1258 //   - A [MxK]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1259 //   - B^T [NxK]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)

```

```

1260 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1261 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1262 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1263 //
1264 // WGMMA (Warp Group MMA, Hopper+):
1265 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1266 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1267 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1268 // threads) 做计算
1269 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1270
1271 // =====
1272 // Phase 7b-1: MMA PTX 宏定义
1273 // =====
1274
1275 // ---- gmem → smem: cp.async ----
1276 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3):
1277 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread)。
1278 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N)。
1279 // N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1280 // - wait_all: 等价于 commit_group + wait_group 0。
1281 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1282 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1283 #define CP_ASYNC_WAIT_GROUP(n) \
1284     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1285
1286 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1287 #define CP_ASYNC_CG(dst, src, bytes) \
1288     asm volatile( \
1289         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1290         "l"(src), "n"(bytes))
1291
1292 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1293 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1294 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1295 // aligned: 要求 128-bit 对齐, trans: 转置加载
1296 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1297     asm volatile( \
1298         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1299         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1300         : "r"(addr))
1301
1302 #define LDMATRIX_X2(R0, R1, addr) \
1303     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1304         : "=r"(R0), "=r"(R1) \
1305         : "r"(addr))
1306
1307 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1308 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1309 #define LDMATRIX_X2_T(R0, R1, addr) \
1310     asm volatile( \
1311         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1312         : "=r"(R0), "=r"(R1) \
1313         : "r"(addr))
1314
1315 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
1316 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1317 // row.col: A 是 row-major, B 是 col-major
1318 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1319 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1320 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1321 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1322     asm volatile(

```



```

1323     "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3}, " \
1324     "%4, %5}, {%6, %7}, {%8, %9};\n" \
1325     : "=r"(RD0), "=r"(RD1) \
1326     : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1327     "r"(RC1))
1328
1329 // ---- MMA 辅助函数 ----
1330 // div_ceil: 整数除法向上取整
1331 #define HOST_DEVICE_INLINE __device__ __host__ inline
1332 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1333     return (a % b != 0) ? (a / b + 1) : (a / b);
1334 }
1335
1336 // =====
1337 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1338 // =====
1339 // 面试重点 — Tile Hierarchy:
1340 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1341 //   MMA Tile (more warps): 2x4=8 个 MMA atom → [2x16, 8x4]=[32,32]
1342 //   VAL Tile (more values): 4x4=16 expand → [32x4, 32x4]=[128,128]
1343 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1344 //           → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1345 //
1346 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1347 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1348 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1349 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1350 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1351 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1352 //
1353 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1354 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1355 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1356 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1357 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1358 //
1359 // ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
1360 //   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
1361 //   (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
1362 //   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
1363 //
1364 //   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
1365 //   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
1366 //
1367 //       N / bx → 0          1          2          3          ...
1368 //       M ↓
1369 //       by = 0  |  SM0   |  SM1   |  SM2   |  SM3   |  ...
1370 //               |  A 0,B0 |  A 0,B1 |  A 0,B2 |  A 0,B3 |
1371 //               +-----+
1372 //
1373 //   同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
1374 //   推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
1375 //   L2 更容易被其他 B tile 挤占。
1376 //
1377 //   Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
1378 //   下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:
1379 //
1380 //       N / local x → 0          1
1381 //       M ↓
1382 //       by = 0  |  SM0   |  SM1   |  → A 0; B^T 0, B^T 1
1383 //               +-----+
1384 //       by = 1  |  SM2   |  SM3   |  → A 1; B^T 0, B^T 1
1385 //               +-----+

```

```

1386 //
1387 // 对于这个 2x2 窗口：同一行横向复用 A (SM0/SM1、SM2/SM3)，同一列纵向复用 B
1388 // (SM0/SM2、SM1/SM3)。真实 grid_x 不必等于 2，但缩小它会缩短再次访问相同 bx 的距离。
1389 // 此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会，不保证 CTA 的 z 顺序、
1390 // 缓存驻留或 L2 hit。
1391 //
1392 // Launch (kBlockSwizzle=false, 当前默认路径)：
1393 // grid = (ceil(N / BN), ceil(M / BM), 1), blockIdx.x 直接就是 N-tile 编号 bx。
1394 // Launch (kBlockSwizzle=true, 需由 launch 端显式构造 3D grid)：
1395 // tiles_n = ceil(N / BN), S = ceil(N / 2048)
1396 // grid = (ceil(tiles_n / S), ceil(M / BM), S)
1397 // bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
1398 // 例如 N=8192、BN=128: tiles_n=64, S=4, grid.x=16; z=0/1/2/3 分别覆盖
1399 // bx=0..15/16..31/32..47/48..63, 即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
1400 // grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
1401 // 对齐; ceil 只保证逻辑 tile 编号不遗漏，并不使部分尾 tile 变为安全。
1402 // Block: (256, 1, 1), 8 warps
1403 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1404 // =====
1405 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1406           const int kMmaN = 8,           // MMA atom N dim
1407           const int kMmaK = 16,          // MMA atom K dim, also BK tile = kMmaK
1408           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1409           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1410           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1411           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1412           const int kStages = 3,         // cp.async pipeline depth
1413           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1414 __global__ void __launch_bounds__(256)
1415   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1416   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1417   // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
1418   // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
1419   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1420   const int by = blockIdx.y;
1421   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1422   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1423   constexpr int BK = kMmaK; // 16
1424
1425   // Dynamic shared memory: kStages 个 stage 的 A 和 B
1426   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1427   // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1428   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1429   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1430   extern __shared__ half smem[];
1431   half *s_a = smem;
1432   half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1433   constexpr int s_a_stage_offset = BM * BK; // 128*16
1434   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major
1435   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1436   const int warp_id = tid / kWarpSize; // 0~7
1437   const int lane_id = tid % kWarpSize; // 0~31
1438   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1439   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0~MMA7的
1440   //
1441   //           N direction (warp_n)
1442   //           0         1         2         3
1443   // M direction (warp_m) 0  MMA0      MMA2      MMA4      MMA6
1444   //                       1  MMA1      MMA3      MMA5      MMA7
1445   // MMA的排布方式不是唯一的, 现在这样排是因为结合MMA/VAL Tile之后, 刚好能覆盖整个
1446   // C Tile[128,128]。只要调整MMA/VAL Tile, 这里的排布方式也可以跟着调整。要注意的
1447   // 是, MMA0-7逻辑上是可以认为是并行执行的, 各自的计算结果累计加到对应的C Tile位置上。
1448   const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1449   const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)

```



```

1449 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1450 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1451 int load_smem_a_m = tid / 2; // 0~127
1452 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1453 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1454 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1455 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1456 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1457 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1458     return;
1459
1460 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1461 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1462 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1463 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1464 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1465
1466 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1467 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1468 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1469
1470 // 预加载前 (kStages-1) 个 stage
1471 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1472 #pragma unroll
1473 for (int k = 0; k < (kStages - 1); ++k) {
1474     int load_gmem_a_k = k * BK + load_smem_a_k;
1475     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1476     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1477         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1478     );
1479     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1480
1481     int load_gmem_b_k = k * BK + load_smem_b_k;
1482     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1483     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1484     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1485         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1486     );
1487     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1488
1489     CP_ASYNC_COMMIT_GROUP();
1490 }
1491
1492 CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1493 __syncthreads();
1494
1495 // 统一循环: k 从 0 开始, 每次迭代负责 tile k (加载 + 计算合并为单循环)
1496 const int NUM_K_TILES = div_ceil(K, BK);
1497 // 此处不用pragma unroll, 因为 K 不是编译期常量, 因此NUM_K_TILES 也不是编译期常量
1498 for (int k = 0; k < NUM_K_TILES; ++k) {
1499     int smem_sel = k % kStages; // 计算 tile k 的 stage
1500     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1501
1502     // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经
1503     // loaded了(kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1504     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1505     // (row-major, 内维连续的是 K)
1506     if (k + kStages - 1 < NUM_K_TILES) {
1507         int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1508         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1509         int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1510         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]

```

```

1512
1513 uint32_t load_smem_a_ptr =
1514     (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1515         load_smem_a_m * BK + load_smem_a_k) *
1516         sizeof(half));
1517 CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1518
1519 uint32_t load_smem_b_ptr =
1520     (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1521         load_smem_b_n * BK + load_smem_b_k) *
1522         sizeof(half));
1523 CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1524 CP_ASYNC_COMMIT_GROUP();
1525 }
1526
1527 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1528 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1529 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1530 uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1531 uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1532
1533 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1534 #pragma unroll
1535 for (int i = 0; i < kValTileM; ++i) {
1536     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1537     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1538     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1539     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1540     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1541     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1542     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1543     uint32_t lane_smem_a_ptr =
1544         (smem_a_base_ptr +
1545             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1546             sizeof(half));
1547     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1548 }
1549
1550 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1551 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1552 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1553 #pragma unroll
1554 for (int j = 0; j < kValTileN; ++j) {
1555     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1556     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1557     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1558     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1559     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1560     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1561     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1562     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1563     uint32_t lane_smem_b_ptr =
1564         (smem_b_base_ptr +
1565             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1566             sizeof(half));
1567     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1568 }
1569
1570 // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次, 在N方向重复kValTileN(4)次
1571 #pragma unroll
1572 for (int i = 0; i < kValTileM; ++i) {
1573     #pragma unroll
1574     for (int j = 0; j < kValTileN; ++j) {

```

```

1575     MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1576             RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1577             RB[j][0], RB[j][1], // B fragment
1578             RC[i][j][0], RC[i][j][1]);
1579 }
1580 }
1581
1582 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1583 if (k + kStages - 1 < NUM_K_TILES) {
1584     CP_ASYNC_WAIT_GROUP(kStages - 2);
1585 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1586     CP_ASYNC_WAIT_GROUP(0);
1587 }
1588 __syncthreads();
1589 }
1590
1591 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1592 {
1593     for (int i = 0; i < kValTileM; ++i) {
1594         // RC[kValTileM][kValTileN][2] 中 RC[...] [...] [2] 中保存了2个 uint32
1595         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...] [...] [0]
1596         // 和RC[...] [...] [1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1597         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1598         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1599         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1600         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1601         // =====
1602         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1603         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1604         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1605         //
1606         //      cols: c0-1    c2-3    c4-5    c6-7
1607         //      r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  |
1608         //      r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1609         //      r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1610         //      ...
1611         //      r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1612         //      r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  |
1613         //      r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1614         //      r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1615         //      ...
1616         //      r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} |
1617         //
1618         // Within a 4-lane group (e.g. T0-T3 for row 0):
1619         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1620         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1621         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1622         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store。
1623         // =====
1624 #pragma unroll
1625         for (int j = 0; j < kValTileN; ++j) {
1626             RC0[j][0] = RC[i][j][0];
1627             RC1[j][0] = RC[i][j][1];
1628             RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1629             RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1630             RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1631             RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1632             RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1633             RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1634         }
1635         // 每 4 个 lane 中只有 lane 0 做 128-bit store
1636         // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行):
1637         //

```

```

1638 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1639 // |-----|-----|-----|-----|
1640 // | 0~3   | 0       | row 0   | row 8   |
1641 // | 4~7   | 1       | row 1   | row 9   |
1642 // | 8~11  | 2       | row 2   | row 10  |
1643 // | 12~15 | 3       | row 3   | row 11  |
1644 // | 16~19 | 4       | row 4   | row 12  |
1645 // | 20~23 | 5       | row 5   | row 13  |
1646 // | 24~27 | 6       | row 6   | row 14  |
1647 // | 28~31 | 7       | row 7   | row 15  |
1648 // |-----|-----|-----|-----|
1649 if (lane_id % 4 == 0) {
1650     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1651     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
1652     // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}, 对应 RC0/RC1 (+8) 的行号, 表示2个8x8矩阵的行号
1653     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
1654 #pragma unroll
1655     for (int j = 0; j < kValTileN; ++j) {
1656         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1657         int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
1658         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
1659         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
1660         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
1661         // 128-bit store: 一次写入 8 个 half
1662         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1663             *reinterpret_cast<float4*>(&RC0[j][0]);
1664         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1665             *reinterpret_cast<float4*>(&RC1[j][0]);
1666     }
1667 }
1668 }
1669 }
1670 }
1671
1672 // =====
1673 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1674 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1675 // =====
1676 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除):
1677 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1678 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way)。
1679 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1680 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1681 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1682 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1683 //   rows 8~11: repeat rows 0~3 pattern
1684 //   rows 12~15: repeat rows 4~7 pattern
1685 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free)。
1686 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1687 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1688 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1689 //
1690 // 参考: LeetCode/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1691 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1692
1693 // ---- Swizzle 辅助函数 ----
1694
1695 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict。
1696 // i: row index; j: col index.
1697 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1698 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1699 // 用位运算展开 (kStep=8): (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1700 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0

```

```

1701 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1702 template <const int kColStride = 16, const int kStep = 8>
1703 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1704     // for col_stride > 16, we have to permute it using col major ZigZag order.
1705     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1706     static_assert(kColStride <= 16, "kColStride must <= 16");
1707     // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1708     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1709     static_assert(kColStride % kStep == 0,
1710         "kColStride must be multiple of kStep.");
1711     if constexpr (kStep == 8) {
1712         // j >> 3: 表示8个half(=16 bytes)数据为一个chunk; i >> 2: 表示4行为一组
1713         // kColStride >> 3: 按照kStep=8计算chunk idx, kColStride=16 -> {0, 1}
1714         // 最后的 << 3: 将chunk idx转换为实际的列偏移量 (8个half/chunk), {0, 8}
1715         return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1716     } else {
1717         static_assert(kStep == 4);
1718         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1719     }
1720 }
1721
1722 // swizzle_A: A 矩阵专用封装 (kMmaAtomK=16, kStep=8)。
1723 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1724 // 8个half=16 bytes=128 bits=4 banks (32 bits/bank) 组成一个phase,
1725 // 触发一次合并的memory transaction. 这里的col=16的swizzle, 相当于
1726 // TMA中的SWIZZLE_32B pattern.
1727 // -----
1728 // -col 0~16, step 8--
1729 // -----
1730 // | row 0 | (0, 8) |
1731 // | row 1 | (0, 8) |
1732 // | row 2 | (0, 8) |
1733 // | row 3 | (0, 8) |
1734 // -----
1735 // | row 4 | (8, 0) |
1736 // | row 5 | (8, 0) |
1737 // | row 6 | (8, 0) |
1738 // | row 7 | (8, 0) |
1739 // -----
1740 // | row 8 | (0, 8) |
1741 // | row 9 | (0, 8) |
1742 // | row 10 | (0, 8) |
1743 // | row 11 | (0, 8) |
1744 // -----
1745 // | row 12 | (8, 0) |
1746 // | row 13 | (8, 0) |
1747 // | row 14 | (8, 0) |
1748 // | row 15 | (8, 0) |
1749 // -----
1750 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1751 template <const int kMmaAtomK = 16>
1752 static __device__ __forceinline__ int swizzle_A(int i, int j) {
1753     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1754 }
1755
1756 // swizzle_B: B 矩阵专用封装 (与 A 相同的 pattern)。
1757 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1758 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8)。
1759 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1760 template <const int kMmaAtomK = 16>
1761 static __device__ __forceinline__ int swizzle_B(int i, int j) {
1762     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1763 }

```

```

1764
1765 // =====
1766 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1767 //           + Register Double Buffering (kValTileK=2, BK=32 统一 tile)
1768 // =====
1769 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1770 //   - kValTileK=2: 每个 BK tile 包含 2 个 kMmaK slice (BK = kMmaK * kValTileK = 32)
1771 //   - BK=32 统一 tile: kMmaK=0 和 kMmaK=1 数据连续存放在同一个 BK=32 宽的 smem tile 中
1772 //   - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换
1773 //   - ldmatrix 与 MMA 计算重叠: 加载 k_step=1 的同时用另一组寄存器做 k_step=0 计算
1774 //
1775 // ★ smem 布局:
1776 //   s_a: [stage0][stage1][stage2], 每个 stage = BM × BK = 128 × 32
1777 //   s_b: 紧接 s_a 之后
1778 //   每个 stage 内 k_step=0 列偏移 0, k_step=1 列偏移 +kMmaK(=16)
1779 //
1780 // ★ 寄存器双缓冲时间线 (每 k 迭代内):
1781 //   Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)
1782 //   Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1783 //   Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1784 //   Step 4: adaptive wait + __syncthreads
1785 //   Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1786 //
1787 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
1788 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1789 // Block: (256, 1, 1), 8 warps
1790 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1791           const int kMmaN = 8,           // MMA atom N dim
1792           const int kMmaK = 16,          // MMA atom K dim
1793           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1794           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1795           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1796           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1797           const int kValTileK = 2,       // MMA_K slices per BK tile, BK = kMmaK*2 = 32
1798           const int kStages = 3,         // cp.async pipeline depth
1799           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1800 __global__ void __launch_bounds__(256)
1801   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1802   static_assert(kValTileK == 2, "Only support kValTileK=2 for register double buffering");
1803   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1804   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1805   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1806   const int by = blockIdx.y;
1807   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1808   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1809   constexpr int BK = kMmaK * kValTileK;             // 16*2=32
1810
1811   // kStages stages, each with BM×BK for A, BN×BK for B
1812   // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK=32 宽 tile 中
1813   extern __shared__ half smem[];
1814   half *s_a = smem;
1815   half *s_b = smem + kStages * BM * BK;
1816   constexpr int s_a_stage_offset = BM * BK;
1817   constexpr int s_b_stage_offset = BN * BK;
1818
1819   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1820   const int warp_id = tid / kWarpSize;
1821   const int lane_id = tid % kWarpSize;
1822   const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1823   const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1824
1825   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1826   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)

```



```

1827 // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 虽然BK=16*2=32, 在后续的kValTileK循环中
1828 // 会加上 k_step*kMmaK=0/16, 最终得到 smem 中的列偏移 0/8/16/24, 正好覆盖 BK=32
1829 int load_smem_a_m = tid / 2; // 0~127
1830 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1831 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1832 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1833 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1834 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1835 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1836     return;
1837
1838 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1839 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1840 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1841 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1842 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1843
1844 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1845 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1846
1847 // Prefetch (kStages-1) stages: load all k_step slices for each early stage
1848 #pragma unroll
1849 for (int k = 0; k < (kStages - 1); ++k) {
1850 #pragma unroll
1851     for (int k_step = 0; k_step < kValTileK; ++k_step) {
1852         int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
1853         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1854         int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
1855         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1856
1857         uint32_t load_smem_a_ptr =
1858             (smem_a_base_ptr +
1859              (k * s_a_stage_offset + load_smem_a_m * BK +
1860               k_step * kMmaK +
1861               swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1862              sizeof(half));
1863         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1864
1865         uint32_t load_smem_b_ptr =
1866             (smem_b_base_ptr +
1867              (k * s_b_stage_offset + load_smem_b_n * BK +
1868               k_step * kMmaK +
1869               swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1870              sizeof(half));
1871         CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1872     }
1873     CP_ASYNC_COMMIT_GROUP();
1874 }
1875
1876 CP_ASYNC_WAIT_GROUP(kStages - 2);
1877 __syncthreads();
1878
1879 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1880 uint32_t RA[2][kValTileM][4];
1881 uint32_t RB[2][kValTileN][2];
1882 int reg_st_idx = 0; // write target for ldmatrix
1883 int reg_ld_idx = 1; // read source for MMA
1884
1885 // Initial ldmatrix: load stage 0, S → R, k_step=0 (0~15列) → reg[0]
1886 // 此时, reg_st_idx = 0, 对0位置的寄存器buffer做初始化。
1887 #pragma unroll
1888 for (int i = 0; i < kValTileM; ++i) {
1889     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;

```



```

1890     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1891     int lane_smem_a_k = (lane_id / 16) * 8;
1892     uint32_t lane_smem_a_ptr =
1893         (smem_a_base_ptr +
1894          (0 * s_a_stage_offset + lane_smem_a_m * BK +
1895           swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1896            sizeof(half));
1897     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1898                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1899                 lane_smem_a_ptr);
1900 }
1901 #pragma unroll
1902 for (int j = 0; j < kValTileN; ++j) {
1903     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1904     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1905     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1906     uint32_t lane_smem_b_ptr =
1907         (smem_b_base_ptr +
1908          (0 * s_b_stage_offset + lane_smem_b_n * BK +
1909           swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1910            sizeof(half));
1911     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1912                 lane_smem_b_ptr);
1913 }
1914
1915 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
1916 // BK = kMmaK * kValTileK = 32, 每个 k 迭代覆盖 BK=32 个 K 元素
1917 const int NUM_K_TILES = div_ceil(K, BK);
1918 for (int k = 0; k < NUM_K_TILES; ++k) {
1919     int smem_sel = k % kStages;
1920     int smem_sel_next = (k + kStages - 1) % kStages;
1921
1922     // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
1923     if (k + kStages - 1 < NUM_K_TILES) {
1924 #pragma unroll
1925         for (int k_step = 0; k_step < kValTileK; ++k_step) {
1926             int load_gmem_a_k =
1927                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
1928             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1929             int load_gmem_b_k =
1930                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
1931             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1932
1933             uint32_t load_smem_a_ptr =
1934                 (smem_a_base_ptr +
1935                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1936                   k_step * kMmaK +
1937                  swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1938                   sizeof(half));
1939             CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1940
1941             uint32_t load_smem_b_ptr =
1942                 (smem_b_base_ptr +
1943                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1944                   k_step * kMmaK +
1945                  swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1946                   sizeof(half));
1947             CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1948         }
1949         CP_ASYNC_COMMIT_GROUP();
1950     }
1951
1952     // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替

```

```

1953 // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
1954 // 每轮: flip->ldmatrix next k_step->reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
1955
1956 #pragma unroll
1957 for (int k_step = 0; k_step < kValTileK; ++k_step) {
1958     reg_st_idx ^= 1; // 0 -> 1; 1 -> 0
1959     reg_ld_idx ^= 1; // 1 -> 0; 0 -> 1
1960
1961     if (k_step + 1 < kValTileK) {
1962         int smem_k_offset = (k_step + 1) * kMmaK;
1963 #pragma unroll
1964         for (int i = 0; i < kValTileM; ++i) {
1965             int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1966             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1967             int lane_smem_a_k = (lane_id / 16) * 8;
1968             uint32_t lane_smem_a_ptr =
1969                 (smem_a_base_ptr +
1970                 (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1971                 smem_k_offset +
1972                 swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1973                 sizeof(half));
1974             LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1975                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1976                 lane_smem_a_ptr);
1977         }
1978 #pragma unroll
1979         for (int j = 0; j < kValTileN; ++j) {
1980             int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1981             int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1982             int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1983             uint32_t lane_smem_b_ptr =
1984                 (smem_b_base_ptr +
1985                 (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1986                 smem_k_offset +
1987                 swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1988                 sizeof(half));
1989             LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1990                 lane_smem_b_ptr);
1991         }
1992     }
1993
1994 #pragma unroll
1995     for (int i = 0; i < kValTileM; ++i) {
1996 #pragma unroll
1997         for (int j = 0; j < kValTileN; ++j) {
1998             MMA16816(RC[i][j][0], RC[i][j][1],
1999                 RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
2000                 RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
2001                 RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
2002                 RC[i][j][0], RC[i][j][1]);
2003         }
2004     }
2005 }
2006
2007 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
2008 if (k + kStages - 1 < NUM_K_TILES) {
2009     CP_ASYNC_WAIT_GROUP(kStages - 2);
2010 } else {
2011     CP_ASYNC_WAIT_GROUP(0);
2012 }
2013 __syncthreads();
2014
2015 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0

```

```

2016     if (k + 1 < NUM_K_TILES) {
2017         smem_sel = (k + 1) % kStages; // old smem_sel + 1
2018 #pragma unroll
2019         for (int i = 0; i < kValTileM; ++i) {
2020             int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2021             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2022             int lane_smem_a_k = (lane_id / 16) * 8;
2023             uint32_t lane_smem_a_ptr =
2024                 (smem_a_base_ptr +
2025                  (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2026                   swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2027                   sizeof(half));
2028             LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2029                         RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2030                         lane_smem_a_ptr);
2031         }
2032 #pragma unroll
2033         for (int j = 0; j < kValTileN; ++j) {
2034             int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2035             int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2036             int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2037             uint32_t lane_smem_b_ptr =
2038                 (smem_b_base_ptr +
2039                  (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2040                   swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2041                   sizeof(half));
2042             LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2043                         lane_smem_b_ptr);
2044         }
2045     }
2046 }
2047
2048 // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
2049 // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间, RA[2][4][4]正好可以复用
2050 for (int i = 0; i < kValTileM; ++i) {
2051 #pragma unroll
2052     for (int j = 0; j < kValTileN; ++j) {
2053         RA[0][j][0] = RC[i][j][0];
2054         RA[1][j][0] = RC[i][j][1];
2055         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
2056         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
2057         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
2058         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
2059         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
2060         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
2061     }
2062     if (lane_id % 4 == 0) {
2063         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2064         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2065 #pragma unroll
2066         for (int j = 0; j < kValTileN; ++j) {
2067             int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2068             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2069             int store_gmem_c_addr_0 =
2070                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2071             int store_gmem_c_addr_1 =
2072                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2073             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2074                 *reinterpret_cast<float4*>(&RA[0][j][0]);
2075             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2076                 *reinterpret_cast<float4*>(&RA[1][j][0]);
2077         }
2078     }

```

```

2079 }
2080 }
2081
2082 // =====
2083 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2084 // =====
2085 // 面试要点 (CuTe vs 手写 MMA PTX) :
2086 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2087 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2088 // - 与手写 MMA PTX (Phase 7b) 的对比:
2089 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
2090 //   - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2091 //     cute::copy / cute::gemm 自动展开为高效指令序列
2092 // - 核心抽象 ("CuTe 五要素") :
2093 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2094 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2095 //   3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2096 //   4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2097 //   5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2098 //
2099 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2100 //   MMA Atom (SM80_16x8x16_F16F16F16_TN)
2101 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2102 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2103 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2104 //
2105 // 调参口诀 (面试速记) :
2106 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2107 //   Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2108 //   重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2109 //     1. 连续的 2^M 个元素组成二维空间中的一个基本元素; (元素宽度)
2110 //     2. 连续的 2^S 个基本元素组成一行; (列数)
2111 //     3. 二维空间包含 2^B 行; (行数)
2112 //     4. 对二维坐标做列置换: icol' = irow ^ icol;
2113 //     5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2114 //   因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2115 //   CuTe 的位级实现等价于:
2116 //     offset' = offset ^ ((offset & YYY) >> S),
2117 //     YYY = ((1 << B) - 1) << (M + S)。
2118 //   对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2119 //   二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2120 //   一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2121 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2122 //   EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2123 //   与逻辑 tile 组织; MMA atom 越多, 需要的线程数就越多
2124 //
2125 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2126 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2127 //   - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2128 //   - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2129 //
2130 // ★ 本实现的 Tile 配置:
2131 //   - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2132 //   - MMA atom: SM80_16x8x16_F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2133 //   - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2134 //     BN=256 不是由 "8x2x16" 直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2135 //   - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2136 //     的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2137 //   - Threads: size(MMA{}) = 128 (4 warps × 2x2 EU repeat)。
2138 //   - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2139 //
2140 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2141 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)

```

```

2142 // =====
2143
2144 #if defined(NOTES_V2_ENABLE_CUTE)
2145
2146 #include <cute/tensor.hpp>
2147
2148 // =====
2149 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2150 // =====
2151 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
2152 // -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价  $B$  col-major  $[K \times N]$ )
2153 // - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2154 //
2155 // Pipeline 流程 (stage 是  $K$  tile 的循环缓冲槽, 不是一条完整计算链):
2156 // 1. PREFETCH: 预加载  $kStage-1$  个  $K$  tile ( $G \rightarrow S$  via cp.async), 填满 stage。
2157 // 2. 主循环 over  $K$  tiles:
2158 // a. 从当前 stage 做  $S \rightarrow R$ : cute::copy(...) → ldmatrix;
2159 // b. 对当前  $K$  slice 做 MMA: cute::gemm(...) → mma.sync;
2160 // c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的  $G \rightarrow S$ ;
2161 // d. 当前 tile 的最后一个  $K$  slice 完成后等待并切换 smem stage。
2162 // 3. Epilogue:  $R \rightarrow S$  (UniversalCopy 写入  $C$  scratchpad)  $\rightarrow S \rightarrow G$  (128-bit store)。
2163 //
2164 // 面试对比 — 手写 MMA vs CuTe 代码量:
2165 // 手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2166 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2167 template <typename T, // element type (half)
2168           int BM, int BN, int BK, // block tile: BM=128, BN=256, BK=32
2169           int kStage, // cp.async pipeline depth (2)
2170           typename TiledMMA, // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
2171           typename G2SCopyA, // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2172           typename G2SCopyB, // G→S B: same as G2SCopyA
2173           typename SmemLayoutA, // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2174           typename SmemLayoutB, // smem B: same atom → (BN,BK,kStage)
2175           typename SmemLayoutC, // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2176           typename S2RCopyAtomA, // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
2177           typename S2RCopyAtomB, // S→R B: same as S2RCopyAtomA
2178           typename R2SCopyAtomC, // R→S C: UniversalCopy<int> (32-bit store)
2179           typename S2GCopyAtomC, // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2180           typename S2GCopyC, // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2181           const int BlockSwizzle = 0> // 1 enables 3D grid swizzle for L2 locality
2182 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2183                                           int n, int k) {
2184     using namespace cute;
2185     static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
2186
2187     // 动态 shared memory:  $KStage$  个  $A/B$  tile;  $C$  epilogue 复用当前  $A$  stage 的空间。
2188     extern __shared__ T shm_data[];
2189     T *Ashm = shm_data;
2190     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2191
2192     int idx = threadIdx.x;
2193     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2194     int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2195     int iy = blockIdx.y; //  $M$  方向 block 索引
2196     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2197     //  $M \% BM == 0, N \% BN == 0, K \% BK == 0$ , 才能避免边界 tile 的越界访问或未写回。
2198     if (iy * BM >= m || ix * BN >= n) return;
2199
2200     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2201     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2202     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2203                             make_stride(k, Int<1>{}));
2204     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),

```

```

2205         make_stride(k, Int<1>{}));
2206 Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2207         make_stride(n, Int<1>{}));
2208
2209 // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2210 // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2211 // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2212 Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2213 Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2214 Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2215
2216 // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2217 auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2218 auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2219
2220 // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2221 TiledMMA tiled_mma;
2222 auto thr_mma = tiled_mma.get_slice(threadIdx.x);
2223 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2224 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2225 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2226 clear(tCrD); // 累加器清零
2227
2228 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2229 G2SCopyA g2s_tiled_copy_a;
2230 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idy);
2231 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2232 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2233
2234 G2SCopyB g2s_tiled_copy_b;
2235 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idy);
2236 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2237 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2238
2239 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2240 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2241 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2242 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idy);
2243 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2244 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2245
2246 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2247 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idy);
2248 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2249 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2250
2251 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2252 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2253 #pragma unroll
2254 for (int istage = 0; istage < kStage - 1; ++istage) {
2255     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2256               tAsA_copy(_, _, _, istage));
2257     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2258               tBsB_copy(_, _, _, istage));
2259     cp_async_fence();
2260     ++itile_to_read;
2261     ++ismem_write;
2262 }
2263 cp_async_wait<kStage - 2>();
2264 __syncthreads();
2265
2266 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2267 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。

```



```

2268 // 当前实现使用整除, 要求  $K \% BK == 0$ ; 没有为尾部  $K$  tile 做 predication/padding。
2269 int num_k_tiles = k / BK;
2270 // 循环前预加载首轮数据: 将第一个  $K$  tile 的第 0 个  $K$  slice 从 smem 加载到寄存器。
2271 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2272 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2273
2274 #pragma unroll 1
2275 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2276     // num_k_steps: BK tile 内  $K$  方向的 MMA 迭代次数, 取自 fragment 的第三 mode ( $K$ -mode) 。
2277     //
2278     // 为什么只显式展开  $K$ ? MMA_M 和 MMA_N 去哪了?
2279     //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2280     //   -  $K$  方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix) ,
2281     //     所以  $K$  循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2282     //   - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2283     //     同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2284     //     在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2285     //     无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2286     //       for (i=0; i<kValTileM; ++i)
2287     //         for (j=0; j<kValTileN; ++j)
2288     //           HMM16816(RC[i][j][0], RC[i][j][1], ...);
2289     //
2290     // CUTLASS 推导链 (mma_atom.hpp) :
2291     //   make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMnK
2292     //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2293     //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2294     //     做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化  $K$  维:
2295     //      $K$  被分为  $BK/kMmaPK$  个 perm- $K$  slice
2296     //     随后按 AtomShape_MnK<2> (MMA_K_atom) 做 zipped_divide
2297     //     每个 perm- $K$  含 kMmaEURepeatK 个 atom- $K$  slice
2298     //   → partition_A 选当前线程后,  $K$ -mode size =  $BK / kMmaPK = 32 / 16 = 2$ 
2299     // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1) , BK=32 → num_k_steps=2
2300     int num_k_steps = size<2>(tCrA);
2301
2302 #pragma unroll
2303     for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2304         int k_step_next = (k_step + 1) % num_k_steps;
2305
2306         if (k_step == num_k_steps - 1) {
2307             // 等待下一个  $K$  tile 的 smem 数据就绪, 然后切换到下一个 stage。
2308             cp_async_wait<kStage - 2>();
2309             __syncthreads();
2310             ismem_read = (ismem_read + 1) % kStage;
2311         }
2312
2313         // S→R: 加载下一组 A/B fragment 到寄存器。
2314         //
2315         // cute::copy 原生支持多 mode——理论上可以一次加载全部  $K$  slice:
2316         //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2317         // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**:
2318         //   当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2319         //   让 S→R 延迟与 MMA 计算重叠。
2320         // 如果一次性加载全部  $K$  slice, S→R 和 MMA 就会彻底串行。
2321         cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2322                   tCrA_view(_, _, k_step_next));
2323         cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2324                   tCrB_view(_, _, k_step_next));
2325
2326         if (k_step == 0) {
2327             // G→S: 提交下一个  $K$  tile (整个 BK) 的异步拷贝。
2328             if (itile_to_read < num_k_tiles) {
2329                 cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2330                           tAsA_copy(_, _, _, ismem_write));

```



```

2331         cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2332                   tBsB_copy(_, _, _, ismem_write));
2333         ++itle_to_read;
2334         ismem_write = (ismem_write + 1) % kStage;
2335     }
2336     cp_async_fence();
2337 }
2338
2339 // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2340 // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2341 cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2342 }
2343 }
2344
2345 // Epilogue: D 寄存器 → shared memory → global memory。
2346 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2347 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2348 //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2349 //   只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2350 //   解释这同一段地址。当前 wrapper 的固定配置下:
2351 //       一个 A stage: 128 x 32 = 4096 个 half;
2352 //       C scratchpad: 32 x 32 x 4 = 4096 个 half。
2353 //   两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2354 //   是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2355 //   C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2356 //   检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2357 //
2358 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2359 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2360 // 用它写 C, S2G 用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2361 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2362
2363 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2364 // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2365 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2366 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2367 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2368 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2369 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2370 //
2371 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2372 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2373 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2374 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2375 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2376 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2377 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2378 // 角色, 而不是另一次 memory transfer。
2379 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2380 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2381 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2382 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2383 //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2384 //   → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2385 //   → make_tiled_copy_C 的 tiler = (32, 32)
2386 //   → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2387 //   → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2388 //   → 结果 shape 的 M/N remainder = _1, _1
2389 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2390 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2391 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2392 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2393

```

```

2394 // S2G TiledCopy: shared memory → global memory (128-bit store)
2395 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2396 S2GCopyC s2g_tiled_copy_c;
2397 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2398 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生，因而保留相同的 pipe=4；前者是
2399 // S2G source mapping，后者是 R2S destination mapping。tCgC_s2g 则面向完整
2400 // gD=(BM,BN)=(128,256)，相对于 S2G copy tiler 仍有非平凡的 M/N repetition，
2401 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2402 // 的逻辑 mode，不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2403 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2404 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2405
2406 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode：
2407 //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2408 //   -> Tensor(CPY, (CPY_M, CPY_N))
2409 // 它只重组 Tensor 的 layout，不分配内存、不移动 data()，也不改变元素访问的物理地址。
2410 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系，只是现在可以用一个坐标
2411 // i+j 访问这个组合 mode；因此这里的 group_modes 更准确地说是“合并 mode”，
2412 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2413 // 从 type/shape 结构看，结果确实是 Tensor(CPY, (CPY_M, CPY_N))；但 grouped
2414 // mode 的 cardinality 是两个子 mode 的乘积，因此 size<1>(…) 可作为一个标量
2415 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2416 // 的线性 logical coordinate，不是在 C++ 中对二维 tuple 做加法，更不是 raw
2417 // pointer offset；最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2418 //
2419 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group？
2420 //   - tCrC_r2s：每个线程持有的寄存器 C fragment，原始形状为 (CPY, CPY_M, CPY_N)；
2421 //   - tCgC_s2g：该线程对应的 global-memory C 输出位置，形状与源 Tensor 对齐；
2422 //   - 两者使用相同的 [1,3) 合并后，源/目的 Tensor 仍保持相同的坐标空间，
2423 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2424 //
2425 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode，因为它们还需要显式保留
2426 // shared-memory C scratchpad 的 pipeline 维度；step=size<3>(tCsC_r2s) 表示
2427 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2428 // 按 step 前进，内层 j 选择当前 pipeline slot：寄存器先写入 sC(_,0,0,j)，
2429 // 再从同一个 slot 写回 global memory。
2430 // 当前 kSmemLayoutCBatch=4，因此 step=4。代码未对 i+j 做尾部 predication，
2431 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除；不应将
2432 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2433 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2434 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2435
2436 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数（由 kSmemLayoutCBatch=4 指定）
2437 // 双层循环的语义（注意 step 与 CPY_N 无关）：
2438 //   group_modes<1,3> 之后，tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2439 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N，即该线程需处理的 C fragment 总数。
2440 //   外循环以 step=4 为步长，内循环 j=0..3 每次处理 1 个 fragment，将其写入
2441 //   第 j 号 C scratchpad pipe slot，再读出写回 global。
2442 //
2443 //   这里 step=4 是 scratchpad pipeline 深度，不是 CPY_N。CPY_N 的值取决于
2444 //   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上，
2445 //   通常 ≠ 4。循环之所以成立，是因为 grouped mode 用线性坐标 i+j 遍历整个
2446 //   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向，也不要求 CPY_N == step。
2447 //   唯一前提是 CPY_M * CPY_N 能被 step 整除（当前静态 layout 编译期保证）。
2448 //
2449 //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2450 //   各自推导；因为二者都从同一个 sC (SmemLayoutC) 出发，pipe mode 保持一致，
2451 //   写入和读取命中同一段 shared memory。
2452 #pragma unroll
2453 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2454     // 每轮处理 step 个 fragment，内层 j 选 pipe slot。
2455 #pragma unroll
2456 for (int j = 0; j < step; ++j) {

```

```

2457 // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2458 // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2459 // 成输出元素类型 T 的临时 payload。
2460 //
2461 // cute::copy (无 copy-atom 的普通版) 内部按元素做
2462 // dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2463 // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2464 // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2465 //
2466 // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2467 // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2468 // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2469 // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2470 // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2471 // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2472 // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2473 // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2474 //
2475 // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2476 // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2477 // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2478 // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2479 // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2480 // 保持同源实现的通用 staging 写法。
2481 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2482 cute::copy(tCrC_r2sx(_, i + j), t);
2483 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2484 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2485 //
2486 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2487 // r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>, T>
2488 // → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2489 // → src(t) 与 dst(tCsC_r2s(_, 0, 0, j)) 都是 rank-1 → 直接 copy_atom.call
2490 // → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2491 // 走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2492 // → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2493 //
2494 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2495 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2496 // - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2497 // → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2498 // - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2499 // → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2500 // 计算了当前线程每个 val 在 smem 中的物理 offset
2501 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2502 // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2503 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2504 }
2505 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2506 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2507 __syncthreads();
2508
2509 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2510 #pragma unroll
2511 for (int j = 0; j < step; ++j) {
2512 // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2513 // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2514 // (如 st.global.v4.f32 或 st.global.b128)。
2515 //
2516 // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2517 // s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>, T>
2518 // → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2519 // → src=tCsC_s2g(_, 0, 0, j) 与 dst=tCgC_s2gx(_, i+j) 都是 rank-1

```

```

2520 // → 直接 copy_atom.call(src, dst)
2521 // → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2522 // → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2523 // → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2524 //
2525 // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2526 // - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2527 // 用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2528 // 在 sC (shared memory) 中的物理 offset。
2529 // S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2530 // = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2531 // 因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2532 // M/N remainder。
2533 // - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2534 // → slice. gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2535 // 因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2536 // repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2537 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2538 // 因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2539 cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2540 }
2541 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2542 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2543 __syncthreads();
2544 }
2545
2546 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2547 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2548 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2549 // CuTe 版没有把这些 lane 映射写死在 kernel 源码中: retile_S 表达 accumulator
2550 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2551 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2552 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2553 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2554 }
2555
2556 // =====
2557 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2558 // =====
2559 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2560 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2561 // kernel 接收这些类型的实例 (通常为 struct), 在编译期完成全部映射推导。
2562 //
2563 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2564 //
2565 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2566 // 答: 手写需要:
2567 // 1) 手动计算每线程的 smem 地址偏移
2568 // 2) 手动计算 ldmatrix 的 lane 映射
2569 // 3) 手动插入 swizzle XOR 到每个地址计算点
2570 // 4) 手动做 epilogue 的 warp shuffle 编排
2571 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2572 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。
2573 // =====
2574 template <typename T, const int Stages = 2>
2575 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2576     using namespace cute;
2577
2578     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2579     //
2580     // kernel 用法对照:
2581     // BM=128: local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2582     //          local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)

```



```

2583 // BN=256: local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2584 // local_tile(D, ...) → gD 的列方向
2585 // BK=32: num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2586 // kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2587 // kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2588 auto BM = Int<128>{};
2589 auto BN = Int<256>{};
2590 auto BK = Int<32>{};
2591 auto KStage = Int<Stages>{};
2592 auto kSmemLayoutCBatch = Int<4>{};
2593
2594 // — SmemLayoutA / SmemLayoutB —
2595 // kernel 用法:
2596 // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2597 // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2598 // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2599 //
2600 // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2601 // - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2602 // - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2603 // - B=3: 共有 2^3=8 行。
2604 // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。
2605 // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2606 // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2607 // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2608 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2609 // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2610 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2611 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2612 // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2613 using SmemLayoutAtom = decltype(composition(
2614     Swizzle<3, 3, 3>{},
2615     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2616         make_stride(Int<BK>{}, Int<1>{}))));
2617 using SmemLayoutA = decltype(tile_to_shape(
2618     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2619 using SmemLayoutB = decltype(tile_to_shape(
2620     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2621
2622 // — MMA (TiledMMA) —
2623 // kernel 用法:
2624 // TiledMMA tiled_mma;
2625 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2626 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2627 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2628 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2629 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2630 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2631 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2632 //
2633 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2634 // → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2635 // → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2636 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2637 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2638 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2639 using mma_traits = MMA_Traits<mma_op>;
2640 using mma_atom = MMA_Atom<mma_traits>;
2641 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2642
2643 static constexpr int kMmaEURepeatM = 2;
2644 static constexpr int kMmaEURepeatN = 2;
2645 static constexpr int kMmaEURepeatK = 1;

```

```

2646 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2647 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2648 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2649 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2650
2651 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2652     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2653 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2654 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2655
2656 // — G2SCopyA / G2SCopyB —
2657 // kernel 用法:
2658 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2659 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2660 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2661 // G→S: 128-bit cp.async. ThrLayout{32,4} × ValLayout{1,8}:
2662 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits.
2663 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2664 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2665 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2666 using G2SCopyA = decltype(make_tiled_copy(
2667     g2s_copy_atom{},
2668     make_layout(make_shape(Int<32>{}, Int<4>{}),
2669         make_stride(Int<4>{}, Int<1>{})),
2670     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2671 using G2SCopyB = G2SCopyA;
2672
2673 // — S2RCopyAtomA / S2RCopyAtomB —
2674 // kernel 用法:
2675 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2676 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2677 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2678 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2679 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2680 // 与 MMA fragment 对齐的 shared→register 映射, 无需手动指定 ThrLayout/ValLayout.
2681 using s2r_copy_op = SM75_U32x4_LDSM_N;
2682 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2683 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2684 using S2RCopyAtomA = s2r_copy_atom;
2685 using S2RCopyAtomB = s2r_copy_atom;
2686
2687 // — SmemLayoutC —
2688 // kernel 用法:
2689 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2690 // 复用当前 A stage 的空间作为 C scratchpad
2691 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2692 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2693 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2694 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2695 // 的 kStage K-tile pipeline.
2696 using SmemLayoutAtomC = decltype(composition(
2697     Swizzle<3, 3, 3>{},
2698     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2699         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2700 using SmemLayoutC = decltype(tile_to_shape(
2701     SmemLayoutAtomC{},
2702     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2703
2704 // — R2SCopyAtomC —
2705 // kernel 用法:
2706 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2707 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2708 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad.

```

```

2709 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2710 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2711 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2712
2713 // — S2GCopyC —
2714 // kernel 用法:
2715 // S2GCopyC s2g_tiled_copy_c;
2716 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2717 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared→global wide store。
2718 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2719 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2720 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2721 using S2GCopyC = decltype(make_tiled_copy(
2722     S2GCopyAtomC{},
2723     make_layout(make_shape(Int<32>{}, Int<4>{}),
2724         make_stride(Int<4>{}, Int<1>{}))),
2725     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2726
2727 // — Grid/Block 配置 —
2728 // kernel 用法:
2729 // int idx = threadIdx.x; // 0..127
2730 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2731 // int iy = blockIdx.y; // M 方向 CTA 坐标
2732 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2733 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2734 // M % BM == 0, N % BN == 0, K % BK == 0。
2735 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2736 constexpr int kBlockSwizzle = 0;
2737 constexpr int kSwizzleStride = 2048;
2738 int BX = (N + BN - 1) / BN;
2739 int BY = (M + BM - 1) / BM;
2740 int BZ = kBlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2741 BX = kBlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2742 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2743 dim3 grid(BX, BY, BZ);
2744
2745 // — Dynamic Shared Memory 大小 —
2746 // kernel 用法:
2747 // extern __shared__ T shm_data[];
2748 // T *Ashm = shm_data;
2749 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2750 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2751 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2752 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2753 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2754 // 能放进一个 A stage (128*32 = 4096)。
2755 static constexpr int shm_size_AB =
2756     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2757 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2758 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2759     "C shared memory must fit within one A pipe");
2760 static constexpr int kShmSize =
2761     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2762
2763 cudaFuncSetAttribute(
2764     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2765         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2766         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2767         S2GCopyAtomC, S2GCopyC, kBlockSwizzle>,
2768     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2769
2770 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2771     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,

```



```

2772         S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2773         kBlockSwizzle>
2774     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2775 }
2776
2777 #endif /* NOTES_V2_ENABLE_CUTE */
2778
2779 #if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
2780 // CUDA 13.2 promotes these TMA operations to cuda::ptx. Keep the older
2781 // experimental wrappers so the Hopper path remains buildable on prior toolkits.
2782 __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
2783     #if CUDART_VERSION >= 13020
2784         cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);
2785     #else
2786         cuda::device::experimental::fence_proxy_async_shared_cta();
2787     #endif
2788 }
2789
2790 __device__ __forceinline__ void tma_load_2d(
2791     void *dst, const CUTensorMap *tensor_map, int minor_coord, int major_coord,
2792     cuda::barrier<cuda::thread_scope_block> &barrier) {
2793     #if CUDART_VERSION >= 13020
2794         const int32_t coords[]{minor_coord, major_coord};
2795         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
2796         cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
2797                                         cuda::ptx::space_global, dst, tensor_map,
2798                                         coords, barrier_handle);
2799     #else
2800         cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2801             dst, tensor_map, minor_coord, major_coord, barrier);
2802     #endif
2803 }
2804
2805 __device__ __forceinline__ void tma_arrive_expect_tx(
2806     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
2807     #if CUDART_VERSION >= 13020
2808         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
2809         [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
2810             cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
2811             barrier_handle, bytes);
2812     #else
2813         [[maybe_unused]] auto token =
2814             cuda::device::barrier_arrive_tx(barrier, 1, bytes);
2815     #endif
2816 }
2817 #endif
2818
2819 // =====
2820 // Phase 7d: HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2821 // =====
2822 // 面试要点 (WGMMA vs MMA 对比) :
2823 //   - MMA: warp 级 (32 threads) , 同步执行
2824 //   - WGMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
2825 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4x16=64倍)
2826 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2827 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMA, 通过 barrier 同步
2828 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2829
2830 // ---- WGMMA 辅助函数 ----
2831 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2832 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2833 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行) 。
2834 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N) 。

```

```

2835 // N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3) ,
2836 // 都是 "wait until only N or fewer groups are pending"。
2837 #define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2838 #define WGMMA_COMMIT_GROUP() \
2839     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2840 #define WGMMA_WAIT_GROUP(n) \
2841     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2842
2843 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
2844 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2845 // encoded = (x & 0x3FFFF) >> 4
2846 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2847 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
2848 // 可省略以扩大可表示范围。
2849 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
2850 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2851
2852 // make_smem_desc: 构造 WGMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2853 // 硬件 WGMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2854 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2855 //
2856 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2857 // -----
2858 // | 位域          | 范围      | 大小 | 含义
2859 // |-----|-----|-----|-----
2860 // | start_address  | [0, 14)   | 14   | smem 基址编码
2861 // | (unused)       | [14, 16)  | 2    | -
2862 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2863 // | (unused)       | [30, 32)  | 2    | -
2864 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2865 // | (unused)       | [46, 49)  | 3    | -
2866 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2867 // | (unused)       | [52, 62)  | 10   | -
2868 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2869 // -----
2870 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2871 //
2872 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2873 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2874 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2875 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2876 // - 这是 stride_byte_offset=1024 的来源
2877 //
2878 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2879 // 此处填 16 仅为占位, 编码后为 1。
2880 //
2881 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2882 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2883 //
2884 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2885 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2886 __device__ inline uint64_t make_smem_desc(half *ptr) {
2887     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2888     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址) 。
2889     uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2890
2891     // 从全零开始: 所有位域初始化为 0。
2892     uint64_t desc = 0x0000000000000000;
2893
2894     // — bits [0, 14): start_address —
2895     // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2896     // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2897     // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。

```

```

2898 desc |= SMEM_DESC_ENCODE(addr);
2899
2900 // — bits [16, 30): leading_byte_offset —
2901 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2902 // << 16 将编码值放到 bits [16, 30)。
2903 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1),
2904 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2905 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2906 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2907 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2908 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2909
2910 // — bits [32, 46): stride_byte_offset —
2911 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2912 // << 32 将编码值放到 bits [32, 46)。
2913 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2)。
2914 // 1024 的来源 (K-Major + 128B swizzle + half):
2915 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2916 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2917 // 若为 MN-Major: SB0 是从首列到下一列的偏移 (8 列一组)。
2918 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2919
2920 // — bits [62, 64): layout_type (swizzle mode) —
2921 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2922 // 1 = 128B swizzle mode。
2923 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2924 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2925 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2926 desc |= 1llu << 62;
2927
2928 return desc;
2929 }
2930
2931 // ---- WGMMMA PTX 指令宏 ----
2932 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2933 // m64n128k16: M=64, N=128, K=16。一次 WGMMMA 计算 D[64×128] += A[64×16] × B[16×128]。
2934 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2935 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
2936 // 每线程 64 half = 32 uint32。
2937 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2938 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2939 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2940 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2941 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2942 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2943                                     TransB) \
2944 { \
2945     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2946     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2947     asm volatile( \
2948         "{\n" \
2949         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2950         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2951         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2952         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2953         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2954         "%32, " \
2955         "%33, " \
2956         "%34, %35, %36, %37, %38;\n" \
2957         "}\n" \
2958         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2959         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2960         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \

```

```

2961         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2962         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2963         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2964         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2965         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2966     : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
2967     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2968     "n"(int32_t(TransB)); \
2969 }
2970
2971 // ---- TMA Shared Memory Layout ----
2972 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2973 //
2974 // 每个 stage 包含两块 smem:
2975 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2976 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
2977 //
2978 // ★ 关键区别 — WGMMMA 的 B 物理存储 vs 逻辑视图:
2979 //
2980 // 上层数据: 源矩阵 B^T [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
2981 // 物理存储: TMA 将 B^T 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
2982 //           (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2983 //           TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2984 //           TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2985 // 逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2986 //           通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2987 //           实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2988 //           swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2989 //
2990 // stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
2991 //   128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
2992 //   stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
2993 //                       = 8 rows × (BK=64) halves × 2 bytes = 1024。
2994 //   这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
2995 //
2996 // 与 MMA(TN) 的对比:
2997 //   MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
2998 //   WGMMMA:   smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
2999 //   简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
3000 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
3001     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
3002     // B tile 笔记:
3003     //   - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
3004     //   - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
3005     //     以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
3006     //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
3007     //     但写 BN×BK 更能体现物理存储形态
3008     alignas(128) half B[BN * BK * QSIZE];
3009 };
3010
3011 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
3012 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
3013 //
3014 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
3015 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
3016 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
3017 //
3018 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
3019 // 将 A/B tile 从 HBM 搬到 shared memory。
3020 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
3021 //
3022 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
3023 //   - full[stage]: Producer 发信号表示 stage stage 的数据已就绪, 可被使用

```

```

3024 // - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕, 可以被覆盖
3025 //
3026 // 本节重点理解:
3027 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
3028 // 即可搬运整个 2D tile, 无需所有线程参与。
3029 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
3030 // 3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时,
3031 // Producer 可以搬运 stage (stage+1), 隐藏 HBM-SMEM 延迟。
3032 //
3033 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
3034 //   WGMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
3035 //   K Tile:          BK=64, 每个 K tile 包含 BK/kWmmaK=4 个 WGMMA atom
3036 //   M Tile:          BM=128, 每个 M tile 包含 BM/kWmmaM=2 个 WGMMA atom
3037 //   N Tile:          BN=128, 单个 kWmmaN=128 即可覆盖, 无需在 N 方向分块
3038 //   Block Tile:      C[128,128] = A[128,64] × B[64,128]
3039 //   Threads:         256 = 2 warpgroups × 128 threads/warpgroup
3040 //   Producer(WG0):   128 threads (仅 thread 0 做 TMA 提交)
3041 //   Consumer(WG1):   128 threads = 4 warps (全部参与 WGMMA 和写回)
3042 //
3043 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM-SMEM
3044 // 的延迟被计算完全掩盖。
3045 //
3046 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
3047 // - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
3048 // Block: (256, 1, 1), 2 warpgroups
3049 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
3050 template <const int kWmmaM = 64,           // WGMMA atom M dim (m64n128k16)
3051           const int kWmmaN = 128,         // WGMMA atom N dim
3052           const int kWmmaK = 16,          // WGMMA atom K dim
3053           const int BM = 128,             // block tile M, 2 × kWmmaM
3054           const int BN = 128,             // block tile N, 1 × kWmmaN
3055           const int BK = 64,              // block tile K, 4 × kWmmaK
3056           const int kNumThreads = 256,    // 2 warpgroups × 128 threads
3057           const int kStages = 3,          // TMA pipeline depth (full/empty barriers)
3058           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
3059 __global__ void __launch_bounds__(kNumThreads)
3060   hgemm_wgmma_stages_tn(
3061     int M, int N, int K, half *C,
3062     const CUtensorMap *__restrict__ tensorMapA,
3063     const CUtensorMap *__restrict__ tensorMapB) {
3064   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3065   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
3066   // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
3067   // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
3068   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
3069   // 不展开宿主侧 create_tensor_map 细节。
3070
3071   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
3072   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
3073   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
3074   const int by = blockIdx.y;
3075   constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
3076   // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
3077   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
3078   constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
3079   // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
3080   // 当只有一个 consumer 时, 它负责全部 BM=128 行
3081   constexpr int kWarpgroupM = BM / kNumConsumers; // 128
3082
3083   // 边界检查: 确保当前 block 不超出 M/N 范围
3084   if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3085     return;
3086

```



```

3087 // ---- Shared Memory 分配 ----
3088 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
3089 // TMA + WGMMMA 仅需 __align__(128), 而非 __align__(1024)。原因:
3090 // make_smem_desc() 构造 WGMMMA descriptor 时, start_address 字段
3091 // 完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量,
3092 // 硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移 (等价于
3093 // descriptor 内置的 base_offset 机制)。因此即使 smem 基址未落到
3094 // 1024B 边界, WGMMMA 硬件也能正确读取 TMA 写入的数据。
3095 // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle
3096 // 函数 tma_swizzle_128B(), 该函数无法感知 smem 绝对地址, 硬编码
3097 // 了 phase=0 的假设, 因此必须 __align__(1024) 来保证 phase 确实为 0。
3098 // 从健壮性角度看本 kernel 也应该用 __align__(1024); 这里保留 128 是
3099 // 为了突出两种消费者的特点与对比。
3100 extern __shared__ __align__(128) uint8_t smem_tma_wgmma_ws[];
3101 WgmmaSMem<BM, BN, BK, kStages> &s =
3102     *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages> *>(smem_tma_wgmma_ws);
3103 half *s_a = s.A;
3104 half *s_b = s.B;
3105
3106 // ---- cuda::barrier 初始化 ----
3107 //
3108 // ★ Barrier 机制速查 (arrive / wait 核心语义):
3109 //
3110 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
3111 //
3112 // init(bar, N): 设置 arrive_count = N。
3113 // 含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
3114 //
3115 // arrive(): 线程声明"我已到达此 barrier 点"。
3116 // - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3117 // - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3118 //
3119 // wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3120 // - 即: 等待 phase 翻转。
3121 // - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3122 // (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3123 //
3124 // 典型用法: b.wait(b.arrive())
3125 // - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3126 // - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3127 // - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3128 // - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3129 //
3130 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129):
3131 //
3132 // Producer (1 thread)          Consumer (128 threads)
3133 // -----
3134 //                                C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3135 // ┌ for each k_tile: ─┐          ┌ for each k_tile: ─┐
3136 // │ P1: arrive+wait(empty[q]) │ │ C1: arrive+wait(full[q]) │
3137 // │   ↓ 等 stage q 被消费完   │ │   ↓ 等 TMA 数据就绪   │
3138 // │ P2: TMA(A[q]) + TMA(B[q]) │ │ C2: WGMMMA 计算       │
3139 // │   ↓ 异步拷贝             │ │ C3: wait WGMMMA 完成   │
3140 // │ P3: arrive_tx(full[q])    │ │ C4: arrive(empty[q])  │
3141 // │   ↑ 通知: stage q 数据就绪 │ │   ↑ 通知: stage q 已消费完 │
3142 // └──────────────────┘          └──────────────────┘
3143 //
3144 // 关键不变式 (invariant):
3145 // - Producer 不会覆盖 Consumer 正在读的 stage
3146 // - Consumer 不会读 Producer 还没写完的 stage
3147 // - 通过 full/empty 两个 barrier 的 phase 交替来保证
3148 //
3149 // 每个 stage 有两个 barrier:

```



```

3150 // full[stage]: TMA 数据就绪信号。Producer 发 (arrive_tx) , Consumer 等 (wait) 。
3151 // empty[stage]: Stage 空闲信号。Consumer 发 (arrive) , Producer 等 (wait) 。
3152 //
3153 // arrive_count = 128 (consumer) + 1 (producer) = 129:
3154 // 每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3155 #pragma nv_diag_suppress static_var_with_dynamic_init
3156 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3157 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3158 #pragma nv_diag_default static_var_with_dynamic_init
3159
3160 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3161 const int NUM_K_TILES = div_ceil(K, BK);
3162 const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
3163 const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup
3164
3165 // 初始化 barriers (仅 thread 0 执行)
3166 if (threadIdx.x == 0) {
3167     for (int i = 0; i < kStages; ++i) {
3168         // arrive_count = 129: 128 consumer + 1 producer
3169         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内) ,
3170         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3171         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3172         init(&full[i], kConsumerThreads + 1); // 128 consumer + 1 producer
3173         init(&empty[i], kConsumerThreads + 1); // same
3174     }
3175     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3176     // 对 async proxy (TMA/WGMMMA) 可见。
3177     tma_fence_proxy_async_shared_cta();
3178 }
3179 __syncthreads();
3180
3181 // =====
3182 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3183 // =====
3184 //
3185 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
3186 // - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
3187 // - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
3188 //
3189 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
3190 // Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 } ← 自己的循环
3191 // Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
3192 //
3193 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
3194 // - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
3195 // - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
3196 //
3197 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
3198 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead) 。
3199 // =====
3200 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3201 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
3202 // 只有 wg_tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
3203 // =====
3204 if (wg_idx == 0) {
3205     if (wg_tid == 0) {
3206         // stage: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3207         int stage = 0;
3208         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3209             if (stage == kStages)
3210                 stage = 0;
3211
3212             // Step P1: 等待 stage stage 变为"空" (可被覆盖写入)

```

```

3213 //
3214 // 模式 empty[stage].wait(empty[stage].arrive()):
3215 //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
3216 //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3217 //     128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
3218 //     → phase 立即翻转。
3219 //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3220 //     由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
3221 //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3222 //
3223 // 语义: Producer 说"我准备好写 stage stage 了, Consumer 用完没有?"
3224 //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3225 //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3226 empty[stage].wait(empty[stage].arrive());
3227
3228 // Step P2: 提交 TMA 2D 拷贝指令
3229 // cp_async_bulk_tensor_2d_global_to_shared:
3230 //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3231 //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3232 //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3233 //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3234 //
3235 // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
3236 // 无需线程逐元素搬运, 零寄存器开销。
3237 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3238 tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
3239             full[stage]);
3240
3241 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3242 tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
3243             full[stage]);
3244
3245 // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
3246 //
3247 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3248 //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3249 //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3250 //   - Phase 翻转条件:
3251 //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3252 //       (b) 所有声明的 async 字节已写入 smem
3253 //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3254 //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3255 tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
3256 }
3257 }
3258 }
3259 // =====
3260 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3261 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3262 // 所有 128 个线程 (4 warps) 全部参与。
3263 // =====
3264 else {
3265 // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3266 //
3267 // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3268 // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[stage].wait()
3269 // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3270 //
3271 // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3272 // Producer 后续的 empty[stage].arrive() 作为第 129 次, 触发 phase 翻转。
3273 for (int i = 0; i < kStages; ++i) {
3274     [[maybe_unused]] auto token = empty[i].arrive();
3275 }

```

```

3276 // 累加器寄存器声明
3277 // d[kWarpgroupM / kWmmaM][kWmmaN / 16][4]:
3278 // - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3279 // - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3280 // - d[*][g][*]: N 方向第 g 组 16 列
3281 // - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3282 // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3283 // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3284 uint32_t d[kWarpgroupM / kWmmaM][kWmmaN / 16][4] = {};
3285
3286
3287 int stage = 0;
3288 // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3289 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3290     if (stage == kStages)
3291         stage = 0;
3292
3293     // Step C1: 等待 TMA 数据就绪 (full 信号)
3294     //
3295     // 模式 full[stage].wait(full[stage].arrive()):
3296     // - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3297     // - 当前 phase 累积: 128/129, phase 尚未翻转。
3298     // - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3299     //   贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3300     //
3301     // 语义: Consumer 说"我准备好读 stage stage 了, 数据到了没有?"
3302     full[stage].wait(full[stage].arrive());
3303
3304     // Step C2: 发射 WGMMMA 指令序列
3305     //
3306     // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3307     // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3308     //
3309     // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3310     // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3311     // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3312     // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3313     //
3314     // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3315     // D[64x128] += A[64x16] × B[16x128]
3316     // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3317     // ScaleD=1: 累加。K 维有 BK/kWmmaK=4 次迭代, 每次都 ScaleD=1。
3318     WGMMMA_FENCE();
3319
3320     // M 维迭代: BM/kWmmaM = 128/64 = 2 个 WGMMMA atom
3321     // 每个 atom 处理 64 行 M 方向数据。
3322     #pragma unroll
3323     for (int m = 0; m < kWarpgroupM / kWmmaM; ++m) {
3324         // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
3325         // s_a 布局: [kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
3326         half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWmmaM;
3327
3328         // K 维迭代: BK/kWmmaK = 64/16 = 4 次 WGMMMA (累加)
3329         // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3330         #pragma unroll
3331         for (int k_step = 0; k_step < BK / kWmmaK; ++k_step) {
3332             // 第 k_step 次 WGMMMA:
3333             // A: wgmma_sA + k_step * kWmmaK (A 的 K 维起始位置)
3334             // B: s_b + stage * BK * BN + k_step * kWmmaK (B 的 K 维起始位置)
3335             // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3336             // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3337             // ScaleD=1: 累加 (K 维迭代需要累积结果)
3338             WGMMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWmmaK,

```

```

3339         s_b + stage * BK * BN + k_step * kWmmaK,
3340         1, // Scaled=1: accumulate (不清零)
3341         1, 1, 0, 0);
3342     }
3343 }
3344
3345 // Step C3: 提交并等待 WGMMMA 完成
3346 //
3347 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3348 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3349 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3350 //
3351 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3352 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3353 // * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3354 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
3355 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3356 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3357 WGMMMA_COMMIT_GROUP();
3358 WGMMMA_WAIT_GROUP(0);
3359
3360 // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
3361 //
3362 // 128 个 Consumer 线程在 empty[stage] 上 arrive(), 为 **下一 phase** 积累到达次数。
3363 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3364 //
3365 // 语义: Consumer 说"stage stage 我已经读完了, 你可以放心覆盖了。"
3366 [[maybe_unused]] auto token = empty[stage].arrive();
3367 }
3368
3369 // =====
3370 // Epilogue: 将寄存器中的累加结果写回 global memory C
3371 // =====
3372 //
3373 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3374 //
3375 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3376 // 这些 half 分布在 d[2][8][4] 数组中:
3377 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3378 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3379 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3380 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3381 //
3382 // 其中:
3383 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
3384 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3385 //
3386 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3387 // +-----+-----+
3388 // | reg[0] | reg[2] | <- rows [row, row+8)
3389 // |(col,+2)|(col+8)|
3390 // +-----+-----+
3391 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3392 // |(col,+2)|(col+8)|
3393 // +-----+-----+
3394 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3395 //
3396 // 三维遍历:
3397 // m_it=0: rows 0~63, m_it=1: rows 64~127
3398 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3399 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3400 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3401

```

```

3402 const int lane = wg_tid % 32;
3403 const int warp = wg_tid / 32;
3404 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3405 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3406 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3407 //         分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3408 //         因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3409 //         同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3410 const int row = warp * 16 + lane / 4;
3411 // block_C: 当前 C tile 在 global memory 中的起始地址
3412 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3413 half *block_C = C + by * BM * N + bx * BN;
3414
3415 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3416 // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
3417 #pragma unroll
3418 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3419     int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
3420 #pragma unroll
3421     for (int g = 0; g < kWgmmaN / 16; ++g) {
3422         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3423         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3424         // 左上象限: (row, col)
3425         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) = d[m][g][0];
3426         // 左下象限: (row+8, col)
3427         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
3428         // 右上象限: (row, col+8)
3429         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
3430         // 右下象限: (row+8, col+8)
3431         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
3432     }
3433 }
3434 }
3435 }
3436
3437 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3438 // SM120 不支持 WGMMMA, 但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
3439 // 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
3440 template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
3441     static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
3442     half A[BM * BK * QSIZE];
3443     half B[BN * BK * QSIZE];
3444 };
3445
3446 // tma_swizzle_128B: 计算 128B TMA swizzle 下 smem 中 (row, k) 的物理 K 偏移。
3447 //
3448 // TMA descriptor 使用 CU_TENSOR_MAP_SWIZZLE_128B 时, 硬件会将写入 smem 的
3449 // 数据按 128B 粒度做 XOR 重映射以消除 bank conflict。消费者在 ldmatrix 前必须
3450 // 对地址施加相同的 swizzle 变换, 否则读到的数据与 TMA 写入的物理位置不一致。
3451 //
3452 // 位级公式与 CuTe 对照:
3453 //   swizzle_k = (((k >> 3) ^ (row & 7)) << 3) | (k & 7)
3454 //
3455 // 等价于 CuTe Swizzle<3,4,3>:
3456 //   M=3 → 基本元素宽 2^3=8 个 half (16 bytes)
3457 //   S=4 → 每行 2^4=16 个基本元素 (32 个 half) ...不, BK=64...
3458 //
3459 // 实际让我们从函数出发:
3460 //   - k & 7: 低 3 位保持不变。即 8 个 half (16 bytes) 为一组, 组内顺序不变。
3461 //   - k >> 3: K 被划分为 8 个 chunk (BK=64, 每 chunk 8 个 half)。
3462 //   - row & 7: 取行号的低 3 位 (swizzle 周期为 8 行)。
3463 //   - XOR: 将 chunk 索引与行号的低 3 位做异或。
3464 //

```

```

3465 // ★ swizzle 映射图 (BK=64, 每 row 8 chunk à 8 half; 周期 = 8 rows × 512 half = 1024B) :
3466 //
3467 // 下图表示同一 chunk 索引在各行被映射到的物理 chunk 位置。例如 chunk 0 在
3468 // row 0 位于物理 chunk 0, 在 row 1 被 XOR 到物理 chunk 1, 以此类推。
3469 //
3470 // chunk idx:      0      1      2      3      4      5      6      7
3471 // ───────────────────────────────────────────────────────────────────────────────────
3472 // row 0 (0^c):    0      1      2      3      4      5      6      7      ← 恒等
3473 // row 1 (1^c):    1      0      3      2      5      4      7      6      ← 相邻对交换
3474 // row 2 (2^c):    2      3      0      1      6      7      4      5
3475 // row 3 (3^c):    3      2      1      0      7      6      5      4
3476 // row 4 (4^c):    4      5      6      7      0      1      2      3
3477 // row 5 (5^c):    5      4      7      6      1      0      3      2
3478 // row 6 (6^c):    6      7      4      5      2      3      0      1
3479 // row 7 (7^c):    7      6      5      4      3      2      1      0
3480 //
3481 // — 8 行一周期 —
3482 // row 8 (0^c): 同 row 0 映射; row 9 同 row 1; 依此类推。
3483 //
3484 // 例子——ldmatrix 读取 row 1、逻辑 k=0 处的数据:
3485 // swizzle_k = (((0 >> 3) ^ (1 & 7)) << 3) | (0 & 7) = ((0 ^ 1) << 3) | 0 = 8
3486 // 即 row 1 的 chunk 0 被 TMA 写到了物理 K=8 的位置, ldmatrix 地址需从 K=8 读。
3487 //
3488 // 例子——ldmatrix 读取 row 3、逻辑 k=16 处的数据:
3489 // swizzle_k = (((16 >> 3) ^ (3 & 7)) << 3) | (16 & 7)
3490 //             = ((2 ^ 3) << 3) | 0 = (1 << 3) | 0 = 8
3491 // 即 row 3 的 chunk 2 被映射到物理 chunk 1 (K=8)。
3492 //
3493 // 关键结论: TMA 和 ldmatrix 两侧必须使用相同的 swizzle; 任何一侧不用或用了
3494 // 不同的 pattern, 整个 tile 的输出都会被破坏。128B = 128 BYTES = 64 half
3495 // = 8 chunk × 8 half/chunk = 8 rows × 512 half/row。
3496 __device__ __forceinline__ int tma_swizzle_128B(int row, int k) {
3497     return (((k >> 3) ^ (row & 7)) << 3) | (k & 7);
3498 }
3499
3500 // 消费者 MMA 层级映射参考 hgemmmmma_stages_tn_swizzle。与那个
3501 // 八 warp kernel 不同, 本 warp-specialized 消费者仅拥有四个 warp。
3502 template <const int kMmaM = 16,                // MMA atom M dim (m16n8k16)
3503           const int kMmaN = 8,                // MMA atom N dim
3504           const int kMmaK = 16,               // MMA atom K dim
3505           const int kMmaTileM = 2,            // consumer warps along M, warp tile M = 32
3506           const int kMmaTileN = 2,            // consumer warps along N, warp tile N = 16
3507           const int kValTileM = 4,            // value-repeat along M, BM = 16*2*4 = 128
3508           const int kValTileN = 8,            // value-repeat along N, BN = 8*2*8 = 128
3509           const int kValTileK = 4,            // MMA_K slices per BK tile, BK = 16*4 = 64
3510           const int kStages = 2,              // TMA full/empty pipeline depth
3511           const int kNumThreads = 256,        // 128 producer + 128 consumer threads
3512           const int kBlockSwizzle = 0>        // 1 enables 3D grid swizzle for L2 locality
3513 __global__ void __launch_bounds__(kNumThreads)
3514 hgemmmmma_ws_tn(
3515     int M, int N, int K, half *C,
3516     const CUtensorMap *__restrict__ tensorMapA,
3517     const CUtensorMap *__restrict__ tensorMapB) {
3518
3519     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
3520     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
3521     constexpr int BK = kMmaK * kValTileK;
3522     static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
3523     static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
3524     static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
3525     static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
3526     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3527

```



```

3528 const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
3529 const int by = blockIdx.y;
3530 constexpr int kConsumerThreads = kNumThreads / 2;
3531 static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
3532             "Consumer threads must cover the MMA warp grid");
3533
3534 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3535     return;
3536
3537 // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐,
3538 // 以确保硬件 swizzle phase 从零开始。消费者 tma_swizzle_128B()
3539 // 假设零 phase; 其他对齐 (如 128) 会导致 phase 偏移, 产生不匹配的
3540 // 物理地址, 破坏整个输出。
3541 //
3542 // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128), 而本 kernel
3543 // 必须 __align__(1024)? 核心区别在于消费者如何感知 swizzle phase:
3544 //
3545 // WGMMMA 消费者: 通过 make_smem_desc() 构造 64-bit WGMMMA descriptor,
3546 // 其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
3547 // 1024B 边界的偏移量 ((addr >> 7) & 0x7)。硬件在读取 smem 时会
3548 // 用这个 base_offset 自动补偿 swizzle phase, 因此 WGMMMA 路径不
3549 // 需要 1024 字节对齐也能得到正确数据。
3550 //
3551 // 本 kernel 消费者: 使用 ldmatrix + 手写 tma_swizzle_128B() 来
3552 // 计算 smem 物理地址。这个函数是纯软件公式, 没有任何 base_offset
3553 // 补偿机制。它假设 smem 基址恰好落在 1024B 边界上 (phase=0)。
3554 // 如果基址只满足 128B 对齐, TMA 硬件会以非零 phase 写入数据,
3555 // 而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
3556 //
3557 // 简言之: WGMMMA 用硬件 descriptor base_offset 自动解决 phase 偏移;
3558 // ldmatrix 路径没有等价机制, 必须靠 1024B 对齐来保证 phase=0。
3559 extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
3560 TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3561     *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
3562 half *s_a = s.A;
3563 half *s_b = s.B;
3564
3565 #pragma nv_diag_suppress static_var_with_dynamic_init
3566 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3567 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3568 #pragma nv_diag_default static_var_with_dynamic_init
3569
3570 const int NUM_K_TILES = div_ceil(K, BK);
3571 const int wg_idx = threadIdx.x / kConsumerThreads;
3572 const int wg_tid = threadIdx.x % kConsumerThreads;
3573
3574 if (threadIdx.x == 0) {
3575     for (int stage = 0; stage < kStages; ++stage) {
3576         init(&full[stage], kConsumerThreads + 1);
3577         init(&empty[stage], kConsumerThreads + 1);
3578     }
3579     tma_fence_proxy_async_shared_cta();
3580 }
3581 __syncthreads();
3582
3583 if (wg_idx == 0) {
3584     // 生产者 Warpgroup (wg_idx==0)
3585     if (wg_tid == 0) {
3586         int stage = 0;
3587         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3588             if (stage == kStages)
3589                 stage = 0;
3590

```

```

3591     empty[stage].wait(empty[stage].arrive());
3592
3593     tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
3594               full[stage]);
3595     tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
3596               full[stage]);
3597     tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
3598 }
3599 }
3600 } else {
3601     // 消费者 Warpgroup (wg_idx==1)
3602     // 初始化: 标记所有 stage 为"空" (可被 Producer 写入)
3603     for (int stage = 0; stage < kStages; ++stage) {
3604         [[maybe_unused]] auto token = empty[stage].arrive();
3605     }
3606
3607     const int warp_id = wg_tid / kWarpSize;
3608     const int lane_id = wg_tid % kWarpSize;
3609     // WG1 内形成 2x2 warp grid, warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
3610     const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
3611     const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
3612     uint32_t RA[kValTileM][4];
3613     uint32_t RB[kValTileN][2];
3614     uint32_t RC[kValTileM][kValTileN][2] = {0};
3615
3616     const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
3617     const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
3618     int stage = 0;
3619     for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3620         if (stage == kStages)
3621             stage = 0;
3622
3623         full[stage].wait(full[stage].arrive());
3624         tma_fence_proxy_async_shared_cta();
3625
3626         // 与 cp.async pipeline (hgemm_mma_stages_tn 等) 不同, 这里 ldmatrix +
3627         // mma.sync 之前/后 不需要 __syncthreads, 原因在于 TMA + async proxy 的
3628         // 同步机制与 warp specialization 的线程分工:
3629         //
3630         // 1. 生产者-消费者解耦: TMA 拷贝由 producer (wg_idx==0 的单个线程)
3631         //    通过 tma_load_2d + tma_arrive_expect_tx 提交, 而非所有 256 个线程
3632         //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
3633         //    拷贝都完成后再进入计算; 而 TMA 路径用 cuda::barrier 的 full/empty
3634         //    协议完成 CTA 级同步——full[stage].wait() 返回时, producer 已通过
3635         //    expect_tx 声明了完整的传输字节数, 且硬件保证这些字节已全部写入 smem。
3636         //
3637         // 2. async proxy fence 替代 __syncthreads: fence_proxy_async(space_shared)
3638         //    在 async proxy (TMA 写入通道) 和后续 smem load (ldmatrix 读取通道)
3639         //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作 (TMA)
3640         //    对 fence 之后的所有 smem 读操作 (ldmatrix) 可见。这是一个轻量级
3641         //    proxy 间 ordering, 不需要阻塞线程, 也不会让 warp 等待其他 warp。
3642         //
3643         // 3. 消费者内部 warp 间无依赖: WG1 内的 4 个 warp (2x2 grid) 各自独立
3644         //    读取 smem 中互不重叠的 A/B 区域 (warp_m 和 warp_n 分别索引不同的
3645         //    行/区间)。它们之间不需要读写共享数据, 因此不需要 __syncthreads 来
3646         //    对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令, 保证同一
3647         //    warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。
3648
3649         // 注意: 这里已经没有 NUM_K_TILES 循环了, 因为 K loop load 已经被 WG0 生产者处理了
3650 #pragma unroll
3651         for (int k_step = 0; k_step < kValTileK; ++k_step) {
3652
3653 #pragma unroll

```

```

3654     for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
3655         // kMmaM * kValTileM = 16*4 = 64, 每个消费者 warp 在 M 方向覆盖 64 行。
3656         const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3657         const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
3658         // 与 hgemm_mma_stages_tn_swizzle 不同: k_step * kMmaK 必须放在
3659         // lane_smem_a_k 中传给 tma_swizzle_128B(), 而非像 swizzle kernel
3660         // 那样作为 smem_k_offset 加在 swizzle 外部。原因:
3661         // tma_swizzle_128B 是 128B TMA swizzle, 作用于完整 BK=64,
3662         // swizzle 周期覆盖全部 64 列, k_step=0 和 k_step=1 的 chunk 会
3663         // 被 XOR 交叉混合 → k_step 偏移必须在 swizzle 内部参与 chunk 计算。
3664         // swizzle_A<kMmaK> 作用于 kMmaK=16, 每个 kMmaK slice 独立
3665         // swizzle, slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
3666         const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
3667         const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
3668             (stage * BM * BK + lane_smem_a_m * BK +
3669             tma_swizzle_128B(lane_smem_a_m, lane_smem_a_k)) *
3670             sizeof(half);
3671         LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
3672     }
3673
3674 #pragma unroll
3675     for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
3676         // kMmaN * kValTileN = 8*8 = 64, 每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
3677         const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3678         const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
3679         const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
3680         const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
3681             (stage * BN * BK + lane_smem_b_n * BK +
3682             tma_swizzle_128B(lane_smem_b_n, lane_smem_b_k)) *
3683             sizeof(half);
3684         LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
3685     }
3686
3687 #pragma unroll
3688     for (int i = 0; i < kValTileM; ++i) {
3689 #pragma unroll
3690         for (int j = 0; j < kValTileN; ++j) {
3691             HMMMA16816(RC[i][j][0], RC[i][j][1],
3692                 RA[i][0], RA[i][1], RA[i][2], RA[i][3],
3693                 RB[j][0], RB[j][1],
3694                 RC[i][j][0], RC[i][j][1]);
3695         }
3696     }
3697 }
3698 [[maybe_unused]] auto token = empty[stage].arrive();
3699 }
3700
3701 // Epilogue: 复用 RA[0] 和 RA[1] (已在寄存器中) 作为 shuffle 缓冲,
3702 // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
3703 // lane 各持有两个 uint32 fragment; shuffle 汇聚为每行一个 float4,
3704 // 由 lane 0 发出对齐的 128-bit store。
3705 #pragma unroll
3706     for (int i = 0; i < kValTileM; ++i) {
3707         const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3708         const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
3709 #pragma unroll
3710         for (int j = 0; j < kValTileN; ++j) {
3711             const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3712             RA[0][0] = RC[i][j][0];
3713             RA[1][0] = RC[i][j][1];
3714             RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
3715             RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
3716             RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);

```

```

3717 RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
3718 RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
3719 RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
3720 if (lane_id % 4 == 0) {
3721     const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
3722     const int store_gmem_c_addr_0 =
3723         store_lane_gmem_c_m * N + store_lane_gmem_c_n;
3724     const int store_gmem_c_addr_1 =
3725         (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
3726     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
3727         *reinterpret_cast<float4*>(&RA[0][0]);
3728     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
3729         *reinterpret_cast<float4*>(&RA[1][0]);
3730 }
3731 }
3732 }
3733 }
3734 }
3735 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
3736
3737 #if defined(NOTES_V2_ENABLE_WGMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3738 // ---- Host-side TMA Tensor Map helpers (WGMMA test) ----
3739 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled) :
3740 //   - TMA descriptor 描述 global memory 中矩阵的 shape/stride/dtype,
3741 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3742 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3743 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3744 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3745 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3746 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMA 的 128B swizzle
3747 //
3748 // ★ gmem_prob_stride + 1 的含义:
3749 //   语法层面 — 数组名退化 + 指针算术:
3750 //     gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3751 //     (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3752 //     gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3753 //   语义层面 — 跳过隐式的最内维 stride:
3754 //     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3755 //     stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3756 //     固定等于 sizeof(element_type), 不需要显式传入。
3757 //     对于 tensorRank=2 的 FP16 矩阵:
3758 //       dim 0 (内维, K) : stride = sizeof(half)    ← 隐式, API 不需要
3759 //       dim 1 (外维, M) : stride = sizeof(half)*K  ← 需要传给 API
3760 //     gmem_prob_stride 数组的布局是内维在前:
3761 //       [0] = sizeof(half)                        ← 内维, 不传
3762 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3763 //     所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3764 //
3765 // ★ smem_box_stride 为什么不需要 +1?
3766 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3767 //   最内维 stride 也必须显式传入 (不隐式)。
3768 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3769 //   都是连续存放的, 维度间没有 padding/stride。
3770 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
3771 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3772 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
3773 //   两个 stride 值, 不需要跳过任何一个。
3774 //
3775 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
3776
3777 template <int BlockMajorSize, int BlockMinorSize>
3778 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3779                                             half *gmem_ptr,

```

```

3780         int blocks_height,
3781         int blocks_width) {
3782     void *gmem_address = (void *)gmem_ptr;
3783     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3784                                     (uint64_t)BlockMajorSize * blocks_height,
3785                                     1, 1, 1};
3786     uint64_t gmem_prob_stride[5] = {
3787         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3788     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3789                                    uint32_t(BlockMajorSize), 1, 1, 1};
3790     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3791     CUresult result = cuTensorMapEncodeTiled(
3792         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3793         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3794         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3795         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3796     if (result != CUDA_SUCCESS)
3797         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3798 }
3799
3800 __host__ static inline CUsensorMap *allocate_and_create_tensor_map(
3801     half *src, int blocks_height, int blocks_width) {
3802     CUsensorMap *tma_map_d;
3803     cudaMalloc(&tma_map_d, sizeof(CUsensorMap));
3804     CUsensorMap tma_map_host;
3805     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
3806     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUsensorMap),
3807               cudaMemcpyHostToDevice);
3808     return tma_map_d;
3809 }
3810 #endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */
3811
3812 // =====
3813 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
3814 // =====
3815 // 面试题点 (FlashAttention 算法) :
3816 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3817 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
3818 // 2. FA 三板斧:
3819 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
3820 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3821 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3822 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3823 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3824 //    - 优点: 减少 warp 间通信和 shuffle
3825 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
3826 //    for each K,V tile:
3827 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3828 //        m_new = max(m_old, row_max(S_cur * scale))
3829 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3830 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3831 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3832 //        O_final = O_new / l_final
3833 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3834 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3835 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3836 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3837 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3838 //      都天然同构”的通用结论
3839 //
3840 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3841 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3842 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64

```

```

3843 // Block: (128, 1, 1), kNumThreads=kWarpSize*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
3844 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3845
3846 // ---- 寄存器填充辅助函数 ----
3847 template <typename T, int M, const int N, const int K = 2>
3848 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3849 #pragma unroll
3850     for (int i = 0; i < M; ++i)
3851 #pragma unroll
3852         for (int j = 0; j < N; ++j)
3853 #pragma unroll
3854             for (int k = 0; k < K; ++k)
3855                 R[i][j][k] = val;
3856 }
3857
3858 template <typename T, int M, const int N = 2>
3859 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3860 #pragma unroll
3861     for (int i = 0; i < M; ++i)
3862 #pragma unroll
3863         for (int j = 0; j < N; ++j)
3864             R[i][j] = val;
3865 }
3866
3867 // =====
3868 // FlashAttention-2 Split-Q Kernel (完整实现)
3869 // =====
3870 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3871 //
3872 // Tile 设计 (以 kHeadDim=64 为例) :
3873 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3874 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3875 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3876 //
3877 // 执行流程:
3878 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3879 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3880 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3881 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
3882 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3883 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3884 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3885 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3886 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3887 //   8) 最终 rescale: O_final = (1/l_final)* O_final
3888 //   9) Epilogue: warp shuffle + 128-bit collective store
3889
3890 template <
3891     const int kHeadDim,           // head dim: 32, 64, 128
3892     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
3893     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
3894     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
3895     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
3896     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
3897     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3898     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
3899     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
3900     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
3901     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
3902     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3903     const int kStage,             // pipeline stages for K: 1 or 2
3904     const int kPad>
3905 __global__ void __launch_bounds__(kWarpSize *kMmaTileSeqLenQ *kMmaTileSeqLenK)

```



```

3906     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3907                                   int QKV_seqlen, int QKV_head) {
3908     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3909     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3910     constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3911     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3912     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3913     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3914     const float scale = 1.0f / sqrtf((float)kHeadDim);
3915
3916     // Block indexing
3917     const int QKV_batch_id = blockIdx.y / QKV_head;
3918     const int QKV_head_id = blockIdx.y % QKV_head;
3919     const int Q_tile_id = blockIdx.x;
3920     const int tid = threadIdx.x;
3921     const int warp_id = tid / kWarpSize;
3922     const int lane_id = tid % kWarpSize;
3923     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3924     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3925
3926     // Global memory base offsets for this (batch, head)
3927     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3928     const int Q_gmem_offset =
3929         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3930         kHeadDim;
3931     const int K_gmem_offset = Q_gmem_offset;
3932     const int V_gmem_offset = Q_gmem_offset;
3933     const int O_gmem_offset = Q_gmem_offset;
3934
3935     // Thread-to-smem mapping for cooperative load
3936     int load_smem_Q_Br = tid / (kNumThreads / Br);
3937     int load_smem_Q_d =
3938         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3939     int load_smem_K_Bc = tid / (kNumThreads / Bc);
3940     int load_smem_K_d =
3941         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3942     int load_smem_V_Bc = tid / (kNumThreads / Bc);
3943     int load_smem_V_d =
3944         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3945
3946     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3947     if (load_gmem_Q_Br >= QKV_seqlen)
3948         return;
3949
3950     // ---- Shared memory layout ----
3951     extern __shared__ half smem[];
3952     constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3953     constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3954     half *Q_tile_smem = smem;
3955     half *K_tile_smem = Q_tile_smem + Q_tile_size;
3956     half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3957     // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3958     // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3959
3960     uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3961     uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3962     uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3963
3964     // ---- Online Softmax persistent state ----
3965     // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3966     // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3967     float lane_block_row_max_old[kValTileSeqLenQ][2];
3968     float lane_block_row_sum_old[kValTileSeqLenQ][2];

```

```

3969 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3970 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3971
3972 // ---- Register allocation ----
3973 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3974 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3975 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3976 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3977 // 对单个 m16n8k16 tile 而言：
3978 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3979 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3980 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3981 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3982 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3983 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3984           [2]; // O accumulator (final output)
3985
3986 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3987 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3988
3989 // =====
3990 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
3991 // =====
3992 {
3993     int load_gmem_Q_addr =
3994         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
3995     uint32_t load_smem_Q_ptr =
3996         smem_Q_base_ptr +
3997         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
3998 #pragma unroll
3999     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4000         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
4001     }
4002     CP_ASYNC_COMMIT_GROUP();
4003 }
4004
4005 // =====
4006 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
4007 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
4008 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
4009 // =====
4010 if constexpr (kStage > 1) {
4011 #pragma unroll
4012     for (int stage = 0; stage < (kStage - 1); ++stage) {
4013         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
4014         int load_gmem_K_addr =
4015             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4016         uint32_t load_smem_K_ptr =
4017             smem_K_base_ptr +
4018             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
4019              load_smem_K_d) *
4020             sizeof(half);
4021 #pragma unroll
4022         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4023             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4024         }
4025         CP_ASYNC_COMMIT_GROUP();
4026     }
4027     CP_ASYNC_WAIT_GROUP(kStage - 2);
4028     __syncthreads();
4029 }
4030
4031 // =====

```

```

4032 // Step 3: 外循环 — 沿 K seqLen 迭代 (Tc = ceil(seqLen/Bc))
4033 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
4034 // =====
4035 #pragma unroll 1
4036 for (int tile_K_seqLen = 0; tile_K_seqLen < Tc; ++tile_K_seqLen) {
4037     int smem_sel = tile_K_seqLen % kStage;
4038     int smem_sel_next = (tile_K_seqLen + (kStage - 1)) % kStage;
4039
4040     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
4041     if constexpr (kStage > 1) {
4042         // 只有 kStage>1 才能真正做 K 的 pipeline:
4043         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
4044         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
4045         // Load current V tile (no pipeline for V — one stage is enough)
4046         {
4047             int load_gmem_V_Bc = tile_K_seqLen * Bc + load_smem_V_Bc;
4048             int load_gmem_V_addr =
4049                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
4050             uint32_t load_smem_V_ptr =
4051                 smem_V_base_ptr +
4052                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
4053 #pragma unroll
4054             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4055                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
4056             }
4057             CP_ASYNC_COMMIT_GROUP();
4058         }
4059
4060         // Prefetch next K tile (pipelined)
4061         if ((tile_K_seqLen + 1) < Tc) {
4062             int load_gmem_K_Bc = (tile_K_seqLen + 1) * Bc + load_smem_K_Bc;
4063             int load_gmem_K_addr =
4064                 K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4065             uint32_t load_smem_K_ptr =
4066                 smem_K_base_ptr +
4067                 (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
4068                  load_smem_K_d) *
4069                 sizeof(half);
4070 #pragma unroll
4071             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4072                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4073             }
4074             CP_ASYNC_COMMIT_GROUP();
4075         }
4076     }
4077
4078     // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
4079     fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
4080 #pragma unroll
4081     for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
4082         // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
4083 #pragma unroll
4084         for (int i = 0; i < kValTileSeqLenQ; ++i) {
4085             int warp_smem_Q_Br =
4086                 warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
4087             int lane_smem_Q_Br =
4088                 warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
4089             int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
4090             uint32_t lane_smem_Q_ptr =
4091                 smem_Q_base_ptr +
4092                 (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
4093             LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4094                        lane_smem_Q_ptr);

```

```

4095     }
4096
4097     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
4098     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
4099     #pragma unroll
4100     for (int j = 0; j < kValTileSeqLenK; ++j) {
4101         int warp_smem_K_Bc =
4102             warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
4103         int lane_smem_K_Bc =
4104             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
4105         int lane_smem_K_d =
4106             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
4107         uint32_t lane_smem_K_ptr =
4108             smem_K_base_ptr +
4109             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
4110              lane_smem_K_d) *
4111              sizeof(half);
4112         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
4113     }
4114
4115     // MMA: S[tile] += Q[tile] @ K^T[tile]
4116     #pragma unroll
4117     for (int i = 0; i < kValTileSeqLenQ; ++i) {
4118         #pragma unroll
4119         for (int j = 0; j < kValTileSeqLenK; ++j) {
4120             MMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
4121                     R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
4122                     R_S[i][j][1]);
4123         }
4124     }
4125 } // end loop over d
4126 __syncthreads();
4127
4128 // =====
4129 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
4130 // =====
4131 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
4132 //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
4133 //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
4134 //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
4135 //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
4136 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
4137 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
4138 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
4139 // kValTileSeqLenK tiles.
4140
4141 float lane_row_max_new[kValTileSeqLenQ][2];
4142 float lane_row_sum_new[kValTileSeqLenQ][2];
4143 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
4144 fill_2D_regs<float>, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
4145
4146 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
4147 #pragma unroll
4148 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4149     #pragma unroll
4150     for (int j = 0; j < kValTileSeqLenK; ++j) {
4151         // Extract half values from R_S registers (C matrix fragment layout)
4152         float2 t_reg_S_0 =
4153             __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
4154         float2 t_reg_S_1 =
4155             __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
4156         // S = (Q@K^T) * scale
4157         float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;

```

```

4158     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
4159     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
4160     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
4161 }
4162 // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
4163 // {0,4,8,...,28} hold valid data)
4164 lane_row_max_new[i][0] =
4165     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
4166 lane_row_max_new[i][1] =
4167     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
4168 }
4169
4170 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
4171 // 面试关键点: 这里将 P 写回 R_S 寄存器!
4172 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
4173 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
4174 #pragma unroll
4175 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4176     // m_new = max(m_old, m_cur)
4177     float block_row_max_new_0 =
4178         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
4179     float block_row_max_new_1 =
4180         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
4181
4182 #pragma unroll
4183 for (int j = 0; j < kValTileSeqLenK; ++j) {
4184     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
4185     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
4186
4187     // P = exp(S * scale - m_new), 用 fma 保证精度
4188     t_reg_S_0.x =
4189         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
4190     t_reg_S_0.y =
4191         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
4192     t_reg_S_1.x =
4193         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
4194     t_reg_S_1.y =
4195         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
4196
4197     // Accumulate row sums
4198     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
4199     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
4200
4201     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
4202     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
4203     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
4204 }
4205
4206 // Warp-level reduce sum (kWarpWidth = 4, same as max)
4207 lane_row_sum_new[i][0] =
4208     warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
4209 lane_row_sum_new[i][1] =
4210     warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
4211 }
4212 __syncthreads();
4213
4214 // =====
4215 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
4216 // =====
4217 // Wait for V to be ready before computing P@V
4218 if constexpr (kStage > 1) {
4219     if ((tile_K_seqlen + 1) < Tc) {
4220         CP_ASYNC_WAIT_GROUP(1);

```

```

4221     } else {
4222         CP_ASYNC_WAIT_GROUP(0);
4223     }
4224 } else {
4225     CP_ASYNC_WAIT_GROUP(0);
4226 }
4227 __syncthreads();
4228
4229 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
4230
4231 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
4232 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
4233 #pragma unroll
4234 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
4235     // ldmatrix.x2.trans: load V[Bc,d] with transposition
4236     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
4237     // transposed ldmatrix
4238 #pragma unroll
4239 for (int j = 0; j < kValTileHeadDimV; ++j) {
4240     int warp_smem_V_d =
4241         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4242     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
4243     int lane_smem_V_d = warp_smem_V_d;
4244     uint32_t lane_smem_V_ptr =
4245         smem_V_base_ptr +
4246         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
4247     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
4248 }
4249
4250 // P matrix layout in R_S[i][j][2]:
4251 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
4252 //   | 64x64 | warp_KV 0 |
4253 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
4254 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
4255 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
4256 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
4257 // tile_V_Bc selects which 16-column slice of P to use:
4258 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
4259 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
4260 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
4261 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
4262 // 对应的 MMA A fragment 布局可以这样记:
4263 //   - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
4264 //   - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
4265 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
4266 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
4267 // fragment 都无条件成立的通用结论。layout转换逻辑:
4268 //   C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
4269 int w = tile_V_Bc * 2;
4270 #pragma unroll
4271 for (int i = 0; i < kValTileSeqLenP; ++i) {
4272 #pragma unroll
4273 for (int j = 0; j < kValTileHeadDimV; ++j) {
4274     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
4275         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
4276         R_V[j][0], R_V[j][1], // B fragment = V
4277         R_0[i][j][0], R_0[i][j][1]); // C fragment output
4278     }
4279 }
4280 } // end for tile_V_Bc
4281 __syncthreads();
4282
4283 // =====

```



```

4284 // 3e: Online rescaling —  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$ 
4285 // =====
4286 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{\text{old}} - m_{\text{new}})$  做  $O$  与  $l$  的 rescale
4287 #pragma unroll
4288 for (int i = 0; i < kValTileSeqLenP; ++i) {
4289     float block_row_max_new_0 = lane_row_max_new[i][0];
4290     float block_row_max_new_1 = lane_row_max_new[i][1];
4291     float block_row_sum_new_0 = lane_row_sum_new[i][0];
4292     float block_row_sum_new_1 = lane_row_sum_new[i][1];
4293
4294     float block_row_max_old_0 = lane_block_row_max_old[i][0];
4295     float block_row_max_old_1 = lane_block_row_max_old[i][1];
4296
4297     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
4298     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
4299
4300     // Handle first iteration:  $m_{\text{old}} = -\text{inf}$ , need to use  $m_{\text{new}}$  directly
4301     block_row_max_old_0 =
4302         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
4303     block_row_max_old_1 =
4304         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
4305
4306     float rescale_o_factor_0 =
4307         __expf(block_row_max_old_0 - block_row_max_new_0);
4308     float rescale_o_factor_1 =
4309         __expf(block_row_max_old_1 - block_row_max_new_1);
4310
4311     // Rescale  $O_{\text{old}}$  + Add  $P@V$  in one fused step
4312 #pragma unroll
4313     for (int j = 0; j < kValTileHeadDimV; ++j) {
4314         //  $R_0 / R_D$  与前面的  $R_S$  一样, 都按 MMA C fragment 布局解释:
4315         // reg[0] -> rows 0~7 的 {c0,c1}
4316         // reg[1] -> rows 8~15 的 {c2,c3}
4317         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
4318         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
4319         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4320         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4321
4322         //  $O_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * O_{\text{old}} + P@V$  (fused multiply-add)
4323         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
4324         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
4325         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
4326         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
4327
4328         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4329         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4330     }
4331
4332     // Update  $l$ :  $l_{\text{new}} = \exp(m_{\text{old}} - m_{\text{new}}) * l_{\text{old}} + \text{row\_sum}(P)$ 
4333     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
4334     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
4335     lane_block_row_sum_old[i][0] = __fmaf_rn(
4336         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
4337     lane_block_row_sum_old[i][1] = __fmaf_rn(
4338         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
4339
4340     // Update  $m$ 
4341     lane_block_row_max_old[i][0] = block_row_max_new_0;
4342     lane_block_row_max_old[i][1] = block_row_max_new_1;
4343 }
4344
4345 // Wait for next K tile to be ready in smem before next iteration
4346 if constexpr (kStage > 1) {

```

```

4347     if ((tile_K_seqlen + 1) < Tc) {
4348         CP_ASYNC_WAIT_GROUP(0);
4349     }
4350     __syncthreads();
4351 }
4352 } // end outer loop over K seqlen
4353 __syncthreads();
4354
4355 // =====
4356 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
4357 // =====
4358 #pragma unroll
4359 for (int i = 0; i < kValTileSeqLenP; ++i) {
4360     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
4361     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
4362     #pragma unroll
4363     for (int j = 0; j < kValTileHeadDimV; ++j) {
4364         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4365         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4366         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
4367         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
4368         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
4369         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
4370         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4371         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4372     }
4373 }
4374
4375 // =====
4376 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
4377 // =====
4378 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
4379 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
4380 #pragma unroll
4381 for (int i = 0; i < kValTileSeqLenP; ++i) {
4382     #pragma unroll
4383     for (int j = 0; j < kValTileHeadDimV; ++j) {
4384         uint32_t R_Z[2][4];
4385         R_Z[0][0] = R_D[i][j][0];
4386         R_Z[1][0] = R_D[i][j][1];
4387         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
4388         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
4389         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
4390         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
4391         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
4392         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
4393
4394         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
4395         if (lane_id % 4 == 0) {
4396             int store_warp_regs_0_Br =
4397                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
4398             int store_lane_gmem_0_Br =
4399                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
4400             int store_warp_regs_0_d =
4401                 warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4402             int store_lane_gmem_0_d = store_warp_regs_0_d;
4403             int store_gmem_0_addr_0 = 0_gmem_offset +
4404                 (store_lane_gmem_0_Br + 0) * kHeadDim +
4405                 store_lane_gmem_0_d;
4406             int store_gmem_0_addr_1 = 0_gmem_offset +
4407                 (store_lane_gmem_0_Br + 8) * kHeadDim +
4408                 store_lane_gmem_0_d;
4409             // LDST128BITS = reinterpret_cast<float4*>

```

```

4410     *reinterpret_cast<float4*>(&O[store_gmem_0_addr_0]) =
4411     *reinterpret_cast<float4*>(&R_Z[0][0]);
4412     *reinterpret_cast<float4*>(&O[store_gmem_0_addr_1]) =
4413     *reinterpret_cast<float4*>(&R_Z[1][0]);
4414 }
4415 }
4416 }
4417 }
4418
4419 // =====
4420 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
4421 // =====
4422
4423 static inline void check(cudaError_t err, const char *msg) {
4424     if (err != cudaSuccess) {
4425         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
4426         exit(EXIT_FAILURE);
4427     }
4428 }
4429
4430
4431 static void test_block_reduce(int N) {
4432
4433     srand(42);
4434     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4435     for (int i = 0; i < N; i++)
4436         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4437
4438     // CPU reference: sum of all elements
4439     double ref = 0.0;
4440     for (int i = 0; i < N; i++) ref += (double)h_a[i];
4441
4442     float *d_a, *d_y;
4443     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
4444     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
4445
4446     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
4447     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
4448
4449     dim3 block(128);
4450     dim3 grid((N + 127) / 128);
4451     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
4452     check(cudaGetLastError(), "blockreduce launch");
4453     check(cudaDeviceSynchronize(), "blockreduce sync");
4454
4455     float result;
4456     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
4457
4458     float err = fabsf(result - (float)ref);
4459     printf("| %-42s | %.6e | %-4s |\n", "BlockReduce", err,
4460         err < 1e-2f ? "PASS" : "FAIL");
4461
4462     free(h_a);
4463     cudaFree(d_a); cudaFree(d_y);
4464 }
4465
4466
4467 static void test_dot(int N) {
4468
4469     srand(42);
4470     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4471     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4472     for (int i = 0; i < N; i++) {

```

```

4473     h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4474     h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4475 }
4476
4477 // CPU reference
4478 double ref = 0.0;
4479 for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4480
4481 float *d_a, *d_b, *d_y;
4482 check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4483 check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4484 check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4485
4486 check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4487 check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4488 check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4489
4490 dim3 block(128);
4491 dim3 grid((N + 127) / 128);
4492 dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4493 check(cudaGetLastError(), "dot launch");
4494 check(cudaDeviceSynchronize(), "dot sync");
4495
4496 float result;
4497 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4498
4499 float err = fabsf(result - (float)ref);
4500 printf("| %-42s | %.6e | %-4s |\n", "Dot", err,
4501     err < 1e-2f ? "PASS" : "FAIL");
4502
4503 // ---- Dot Vec4 ----
4504 check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4505 dim3 block_v4(32);
4506 dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
4507 check(cudaGetLastError(), "dot_vec4 launch");
4508 check(cudaDeviceSynchronize(), "dot_vec4 sync");
4509
4510 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4511 float err_v4 = fabsf(result - (float)ref);
4512 printf("| %-42s | %.6e | %-4s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL");
4513
4514 free(h_a); free(h_b);
4515 cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4516 }
4517
4518
4519 static void test_relu(int N) {
4520     srand(42);
4521     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4522     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4523     for (int i = 0; i < N; i++)
4524         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4525
4526     float *d_x, *d_y;
4527     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
4528     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
4529     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4530
4531     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4532     dim3 block256(256);
4533     dim3 grid256((N + 255) / 256);
4534     relu<<<grid256, block256>>>(d_x, d_y, N);
4535     check(cudaGetLastError(), "relu launch");

```

```

4536 check(cudaDeviceSynchronize(), "relu sync");
4537 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4538 float max_err = 0.0f;
4539 for (int i = 0; i < N; i++) {
4540     float expected = fmaxf(0.0f, h_x[i]);
4541     float err = fabsf(h_y[i] - expected);
4542     if (err > max_err) max_err = err;
4543 }
4544 printf("| %-42s | %.6e | %-4s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4545
4546 dim3 block64(64);
4547 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4548 check(cudaGetLastError(), "relu_vec4 launch");
4549 check(cudaDeviceSynchronize(), "relu_vec4 sync");
4550 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4551 max_err = 0.0f;
4552 for (int i = 0; i < N; i++) {
4553     float expected = fmaxf(0.0f, h_x[i]);
4554     float err = fabsf(h_y[i] - expected);
4555     if (err > max_err) max_err = err;
4556 }
4557 printf("| %-42s | %.6e | %-4s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4558
4559 free(h_x); free(h_y);
4560 cudaFree(d_x); cudaFree(d_y);
4561 }
4562
4563
4564 static void test_elementwise(int N) {
4565     srand(42);
4566     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4567     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4568     for (int i = 0; i < N; i++) {
4569         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4570         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4571     }
4572
4573     float *d_a, *d_b, *d_c;
4574     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4575     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4576     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4577     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4578     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4579
4580     dim3 block256(256);
4581     dim3 grid256((N + 255) / 256);
4582     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4583     check(cudaGetLastError(), "eadd launch");
4584     check(cudaDeviceSynchronize(), "eadd sync");
4585     float *h_c = (float *)malloc((size_t)N * sizeof(float));
4586     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4587     float max_err = 0.0f;
4588     for (int i = 0; i < N; i++) {
4589         float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4590         if (err > max_err) max_err = err;
4591     }
4592     printf("| %-42s | %.6e | %-4s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4593
4594     dim3 block64(64);
4595     check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
4596     elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4597     check(cudaGetLastError(), "eadd_vec4 launch");
4598     check(cudaDeviceSynchronize(), "eadd_vec4 sync");

```

```

4599 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4600 max_err = 0.0f;
4601 for (int i = 0; i < N; i++) {
4602     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4603     if (err > max_err) max_err = err;
4604 }
4605 printf("| %-42s | %.6e | %-4s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4606
4607 free(h_a); free(h_b); free(h_c);
4608 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4609 }
4610
4611
4612 static void test_histogram(int N) {
4613     srand(42);
4614     int BINS = 16;
4615     int *h_hist = (int *)calloc(BINS, sizeof(int));
4616     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4617     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4618     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4619     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4620
4621     int *d_idx, *d_hist;
4622     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4623     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4624     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
4625     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4626
4627     dim3 block256(256);
4628     dim3 grid256((N + 255) / 256);
4629     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
4630     check(cudaGetLastError(), "histogram launch");
4631     check(cudaDeviceSynchronize(), "histogram sync");
4632     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
4633
4634     float max_err = 0.0f;
4635     for (int i = 0; i < BINS; i++) {
4636         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4637         if (err > max_err) max_err = err;
4638     }
4639     printf("| %-42s | %.6e | %-4s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4640
4641     free(h_hist); free(h_hist_ref); free(h_idx);
4642     cudaFree(d_idx); cudaFree(d_hist);
4643 }
4644
4645
4646 static void test_merge_attn_states(int num_tokens, int num_heads,
4647                                   int head_size) {
4648
4649     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4650     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4651
4652     float *h_prefix_out = (float *)malloc(out_size);
4653     float *h_suffix_out = (float *)malloc(out_size);
4654     float *h_prefix_lse = (float *)malloc(lse_size);
4655     float *h_suffix_lse = (float *)malloc(lse_size);
4656     float *h_output_ref = (float *)malloc(out_size);
4657
4658     srand(42);
4659     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4660         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4661         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;

```



```

4662 }
4663 for (int i = 0; i < num_heads * num_tokens; i++) {
4664     h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4665     h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4666 }
4667
4668 // CPU reference: 与 kernel 完全相同的浮点计算
4669 for (int t = 0; t < num_tokens; t++) {
4670     for (int h = 0; h < num_heads; h++) {
4671         float p_lse = h_prefix_lse[h * num_tokens + t];
4672         float s_lse = h_suffix_lse[h * num_tokens + t];
4673         p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4674         s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4675
4676         float max_lse = fmaxf(p_lse, s_lse);
4677         p_lse -= max_lse;
4678         s_lse -= max_lse;
4679         float p_se = expf(p_lse);
4680         float s_se = expf(s_lse);
4681         float p_scale = p_se / (p_se + s_se);
4682         float s_scale = s_se / (p_se + s_se);
4683
4684         int head_off = t * num_heads * head_size + h * head_size;
4685         for (int d = 0; d < head_size; d++) {
4686             h_output_ref[head_off + d] =
4687                 h_prefix_out[head_off + d] * p_scale +
4688                 h_suffix_out[head_off + d] * s_scale;
4689         }
4690     }
4691 }
4692
4693 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4694 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
4695 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4696 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4697 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4698 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4699
4700 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4701                 cudaMemcpyHostToDevice),
4702        "merge_attn H2D p_out");
4703 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4704                 cudaMemcpyHostToDevice),
4705        "merge_attn H2D s_out");
4706 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4707                 cudaMemcpyHostToDevice),
4708        "merge_attn H2D p_lse");
4709 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4710                 cudaMemcpyHostToDevice),
4711        "merge_attn H2D s_lse");
4712
4713 int threads_per_head = head_size / 4;
4714 int total_threads = num_tokens * num_heads * threads_per_head;
4715 dim3 block(128);
4716 dim3 grid((total_threads + 127) / 128);
4717 merge_attn_states<<<grid, block>>>(
4718     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4719     num_tokens, num_heads, head_size);
4720 check(cudaGetLastError(), "merge_attn launch");
4721 check(cudaDeviceSynchronize(), "merge_attn sync");
4722
4723 float *h_output = (float *)malloc(out_size);
4724 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),

```

```

4725         "merge_attn D2H");
4726
4727     float max_err = 0.0f;
4728     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4729         float err = fabsf(h_output[i] - h_output_ref[i]);
4730         if (err > max_err) max_err = err;
4731     }
4732     printf("| %-42s | %.6e | %-4s |\n", "MergeAttnStates", max_err,
4733         max_err < 1e-4f ? "PASS" : "FAIL");
4734
4735     // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
4736     h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
4737     h_suffix_lse[0] = 0.0f;
4738     check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4739         cudaMemcpyHostToDevice),
4740         "merge_attn H2D inf lse");
4741     merge_attn_states<<<grid, block>>>(
4742         d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4743         num_tokens, num_heads, head_size);
4744     check(cudaGetLastError(), "merge_attn inf launch");
4745     check(cudaDeviceSynchronize(), "merge_attn inf sync");
4746     check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4747         "merge_attn inf D2H");
4748
4749     // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
4750     float inf_err = 0.0f;
4751     for (int d = 0; d < head_size; d++) {
4752         float err = fabsf(h_output[d] - h_suffix_out[d]);
4753         if (err > inf_err) inf_err = err;
4754     }
4755     printf("| %-42s | %.6e | %-4s |\n", "MergeAttnStates-inf", inf_err,
4756         inf_err < 1e-4f ? "PASS" : "FAIL");
4757
4758     free(h_prefix_out);
4759     free(h_suffix_out);
4760     free(h_prefix_lse);
4761     free(h_suffix_lse);
4762     free(h_output_ref);
4763     free(h_output);
4764     cudaFree(d_prefix_out);
4765     cudaFree(d_suffix_out);
4766     cudaFree(d_prefix_lse);
4767     cudaFree(d_suffix_lse);
4768     cudaFree(d_output);
4769 }
4770
4771
4772 static void test_softmax(int N) {
4773     // Use N = blockDim.x so one block processes all elements as one token.
4774     constexpr int kNumThreads = 256;
4775     if (N != kNumThreads) {
4776         printf("  OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
4777         return;
4778     }
4779
4780     srand(42);
4781     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4782     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4783     for (int i = 0; i < N; i++)
4784         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
4785
4786     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4787     float max_val = -FLT_MAX;

```

```

4788 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4789 double sum_exp = 0.0;
4790 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4791 for (int i = 0; i < N; i++)
4792     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4793
4794 float *d_x, *d_y;
4795 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4796 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4797
4798 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4799
4800 dim3 block(256);
4801 dim3 grid(1); // one token covering all N elements
4802 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4803 check(cudaGetLastError(), "softmax launch");
4804 check(cudaDeviceSynchronize(), "softmax sync");
4805
4806 float *h_y = (float *)malloc((size_t)N * sizeof(float));
4807 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4808
4809 float max_err = 0.0f;
4810 for (int i = 0; i < N; i++) {
4811     float err = fabsf(h_y[i] - h_y_ref[i]);
4812     if (err > max_err) max_err = err;
4813 }
4814 printf("| %-42s | %.6e | %-4s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4815
4816 // ---- Safe Softmax ----
4817 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4818 check(cudaGetLastError(), "safe_softmax launch");
4819 check(cudaDeviceSynchronize(), "safe_softmax sync");
4820 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4821 max_err = 0.0f;
4822 for (int i = 0; i < N; i++) {
4823     float err = fabsf(h_y[i] - h_y_ref[i]);
4824     if (err > max_err) max_err = err;
4825 }
4826 printf("| %-42s | %.6e | %-4s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4827
4828 // ---- Naive Softmax ----
4829 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4830 check(cudaGetLastError(), "naive_softmax launch");
4831 check(cudaDeviceSynchronize(), "naive_softmax sync");
4832 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4833 max_err = 0.0f;
4834 for (int i = 0; i < N; i++) {
4835     float err = fabsf(h_y[i] - h_y_ref[i]);
4836     if (err > max_err) max_err = err;
4837 }
4838 printf("| %-42s | %.6e | %-4s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4839
4840 free(h_x); free(h_y); free(h_y_ref);
4841 cudaFree(d_x); cudaFree(d_y);
4842 }
4843
4844
4845 static void test_rms_norm(int N, int K) {
4846
4847     srand(42);
4848     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4849     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4850     for (int i = 0; i < N * K; i++)

```

```

4851     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4852     float g = 1.5f;    // gain
4853
4854     // CPU reference: y = (x / rms(x)) * g
4855     float epsilon = 1e-5f;
4856     for (int n = 0; n < N; n++) {
4857         double sum_sq = 0.0;
4858         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4859         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4860         for (int k = 0; k < K; k++)
4861             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4862     }
4863
4864     float *d_x, *d_y;
4865     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4866     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4867     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4868
4869     dim3 block(128);
4870     dim3 grid(N);
4871     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4872     check(cudaGetLastError(), "rmsnorm launch");
4873     check(cudaDeviceSynchronize(), "rmsnorm sync");
4874
4875     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4876     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4877
4878     float max_err = 0.0f;
4879     for (int i = 0; i < N * K; i++) {
4880         float err = fabsf(h_y[i] - h_y_ref[i]);
4881         if (err > max_err) max_err = err;
4882     }
4883     printf("| %-42s | %.6e | %-4s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4884
4885     // ---- RMS Norm Vec4 ----
4886     dim3 block_rv4(32);
4887     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4888     check(cudaGetLastError(), "rmsnorm_vec4 launch");
4889     check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4890     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4891     max_err = 0.0f;
4892     for (int i = 0; i < N * K; i++) {
4893         float err = fabsf(h_y[i] - h_y_ref[i]);
4894         if (err > max_err) max_err = err;
4895     }
4896     printf("| %-42s | %.6e | %-4s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4897
4898     free(h_x); free(h_y); free(h_y_ref);
4899     cudaFree(d_x); cudaFree(d_y);
4900 }
4901
4902
4903 static void test_layer_norm(int N, int K) {
4904
4905     srand(42);
4906     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4907     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4908     for (int i = 0; i < N * K; i++)
4909         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4910     float g = 1.5f, b = 0.3f;    // gain and bias
4911
4912     // CPU reference: y = ((x - mean) / std) * g + b
4913     float epsilon = 1e-5f;

```

```

4914 for (int n = 0; n < N; n++) {
4915     double sum = 0.0;
4916     for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4917     float mean = (float)sum / (float)K;
4918     double sum_sq = 0.0;
4919     for (int k = 0; k < K; k++) {
4920         float diff = h_x[n * K + k] - mean;
4921         sum_sq += (double)diff * (double)diff;
4922     }
4923     float std = sqrtf((float)sum_sq / (float)K + epsilon);
4924     for (int k = 0; k < K; k++)
4925         h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
4926 }
4927
4928 float *d_x, *d_y;
4929 check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4930 check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4931 check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4932
4933 dim3 block(128);
4934 dim3 grid(N);
4935 layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4936 check(cudaGetLastError(), "layernorm launch");
4937 check(cudaDeviceSynchronize(), "layernorm sync");
4938
4939 float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4940 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4941
4942 float max_err = 0.0f;
4943 for (int i = 0; i < N * K; i++) {
4944     float err = fabsf(h_y[i] - h_y_ref[i]);
4945     if (err > max_err) max_err = err;
4946 }
4947 printf("| %-42s | %.6e | %-4s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4948
4949 // ---- Layer Norm Vec4 ----
4950 dim3 block_lv4(32);
4951 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4952 check(cudaGetLastError(), "layernorm_vec4 launch");
4953 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4954 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4955 max_err = 0.0f;
4956 for (int i = 0; i < N * K; i++) {
4957     float err = fabsf(h_y[i] - h_y_ref[i]);
4958     if (err > max_err) max_err = err;
4959 }
4960 printf("| %-42s | %.6e | %-4s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
4961
4962 free(h_x); free(h_y); free(h_y_ref);
4963 cudaFree(d_x); cudaFree(d_y);
4964 }
4965
4966 static void test_rope(int seq_len, int N) {
4967
4968     int total_pairs = seq_len * N;
4969     int total_elems = total_pairs * 2;
4970     size_t size = (size_t)total_elems * sizeof(float);
4971
4972     float *h_x = (float *)malloc(size);
4973     float *h_y_ref = (float *)malloc(size);
4974
4975     srand(42);
4976

```

```

4977 for (int i = 0; i < total_elems; i++)
4978     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4979
4980 // CPU reference: 2D rotation for each pair
4981 for (int idx = 0; idx < total_pairs; idx++) {
4982     int token_pos = idx / N;
4983     int token_idx = idx % N;
4984     float x1 = h_x[idx * 2];
4985     float x2 = h_x[idx * 2 + 1];
4986     float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4987     float angle = (float)token_pos * theta;
4988     float cos_v = cosf(angle);
4989     float sin_v = sinf(angle);
4990     h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
4991     h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
4992 }
4993
4994 float *d_x, *d_y;
4995 check(cudaMalloc(&d_x, size), "rope alloc X");
4996 check(cudaMalloc(&d_y, size), "rope alloc Y");
4997 check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
4998
4999 dim3 block(256);
5000 dim3 grid((total_pairs + 255) / 256);
5001 rope<<<grid, block>>>(d_x, d_y, seq_len, N);
5002 check(cudaGetLastError(), "rope launch");
5003 check(cudaDeviceSynchronize(), "rope sync");
5004
5005 float *h_y = (float *)malloc(size);
5006 check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
5007
5008 float max_err = 0.0f;
5009 for (int i = 0; i < total_elems; i++) {
5010     float err = fabsf(h_y[i] - h_y_ref[i]);
5011     if (err > max_err) max_err = err;
5012 }
5013 printf("| %-42s | %.6e | %-4s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL");
5014
5015 free(h_x);
5016 free(h_y);
5017 free(h_y_ref);
5018 cudaFree(d_x);
5019 cudaFree(d_y);
5020 }
5021
5022
5023 static void test_mat_transpose(int row, int col) {
5024
5025     size_t size_in = (size_t)row * col * sizeof(float);
5026     size_t size_out = (size_t)col * row * sizeof(float);
5027
5028     float *h_x = (float *)malloc(size_in);
5029     float *h_y_ref = (float *)malloc(size_out);
5030
5031     srand(42);
5032     for (int i = 0; i < row * col; i++)
5033         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5034
5035     // CPU reference: y[j][i] = x[i][j] (row-major)
5036     for (int i = 0; i < row; i++)
5037         for (int j = 0; j < col; j++)
5038             h_y_ref[j * row + i] = h_x[i * col + j];
5039

```



```

5040 float *d_x, *d_y;
5041 check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
5042 check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
5043 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
5044
5045 dim3 block(16, 16);
5046 dim3 grid((col + 15) / 16, (row + 15) / 16);
5047 mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
5048 check(cudaGetLastError(), "mattrans launch");
5049 check(cudaDeviceSynchronize(), "mattrans sync");
5050
5051 float *h_y = (float *)malloc(size_out);
5052 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
5053
5054 float max_err = 0.0f;
5055 for (int i = 0; i < col * row; i++) {
5056     float err = fabsf(h_y[i] - h_y_ref[i]);
5057     if (err > max_err) max_err = err;
5058 }
5059 printf("| %-42s | %.6e | %-4s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
5060
5061 free(h_x);
5062 free(h_y);
5063 free(h_y_ref);
5064 cudaFree(d_x);
5065 cudaFree(d_y);
5066 }
5067
5068
5069 static void test_mat_transpose_padded(int row, int col) {
5070
5071     size_t size_in = (size_t)row * col * sizeof(float);
5072     size_t size_out = (size_t)col * row * sizeof(float);
5073
5074     float *h_x = (float *)malloc(size_in);
5075     float *h_y_ref = (float *)malloc(size_out);
5076
5077     srand(42);
5078     for (int i = 0; i < row * col; i++)
5079         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5080
5081     // CPU reference: y[j][i] = x[i][j] (row-major)
5082     for (int i = 0; i < row; i++)
5083         for (int j = 0; j < col; j++)
5084             h_y_ref[j * row + i] = h_x[i * col + j];
5085
5086     float *d_x, *d_y;
5087     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
5088     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
5089     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
5090
5091     dim3 block(16, 16);
5092     dim3 grid((col + 15) / 16, (row + 63) / 64);
5093     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
5094     check(cudaGetLastError(), "mattrans_padded launch");
5095     check(cudaDeviceSynchronize(), "mattrans_padded sync");
5096
5097     float *h_y = (float *)malloc(size_out);
5098     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
5099
5100     float max_err = 0.0f;
5101     for (int i = 0; i < col * row; i++) {
5102         float err = fabsf(h_y[i] - h_y_ref[i]);

```

```

5103     if (err > max_err) max_err = err;
5104 }
5105 printf("| %-42s | %.6e | %-4s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL");
5106
5107 free(h_x);
5108 free(h_y);
5109 free(h_y_ref);
5110 cudaFree(d_x);
5111 cudaFree(d_y);
5112 }
5113
5114
5115 static void test_sgemv(int M, int K) {
5116
5117     srand(42);
5118     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
5119     float *h_x = (float *)malloc((size_t)K * sizeof(float));
5120     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5121     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5122     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5123
5124     // CPU reference: y[m] = sum_k A[m,k] * x[k]
5125     for (int m = 0; m < M; m++) {
5126         double sum = 0.0;
5127         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
5128         h_y_ref[m] = (float)sum;
5129     }
5130
5131     float *d_a, *d_x, *d_y;
5132     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
5133     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
5134     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
5135
5136     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
5137     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
5138
5139     dim3 block(32, 4);
5140     dim3 grid((M + 3) / 4);
5141     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
5142     check(cudaGetLastError(), "sgemv launch");
5143     check(cudaDeviceSynchronize(), "sgemv sync");
5144
5145     float *h_y = (float *)malloc((size_t)M * sizeof(float));
5146     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
5147
5148     float max_err = 0.0f;
5149     for (int m = 0; m < M; m++) {
5150         float err = fabsf(h_y[m] - h_y_ref[m]);
5151         if (err > max_err) max_err = err;
5152     }
5153     printf("| %-42s | %.6e | %-4s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
5154
5155     // ---- SGEMV K32 ----
5156     check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
5157     sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
5158     check(cudaGetLastError(), "sgemv_k32 launch");
5159     check(cudaDeviceSynchronize(), "sgemv_k32 sync");
5160
5161     check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
5162
5163     max_err = 0.0f;
5164     for (int m = 0; m < M; m++) {
5165         float err = fabsf(h_y[m] - h_y_ref[m]);

```

```

5166     if (err > max_err) max_err = err;
5167 }
5168 printf("| %-42s | %.6e | %-4s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
5169
5170 // ---- SGEMV K16 ----
5171 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5172 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5173
5174 int K16 = 16;
5175 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
5176 h_x = (float *)malloc((size_t)K16 * sizeof(float));
5177 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5178 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5179 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5180
5181 for (int m = 0; m < M; m++) {
5182     double sum = 0.0;
5183     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
5184     h_y_ref[m] = (float)sum;
5185 }
5186
5187 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
5188 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
5189 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
5190
5191 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
5192 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
5193
5194 dim3 grid_k16((M + 7) / 8);
5195 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
5196 check(cudaGetLastError(), "sgemv_k16 launch");
5197 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
5198
5199 h_y = (float *)malloc((size_t)M * sizeof(float));
5200 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
5201
5202 max_err = 0.0f;
5203 for (int m = 0; m < M; m++) {
5204     float err = fabsf(h_y[m] - h_y_ref[m]);
5205     if (err > max_err) max_err = err;
5206 }
5207 printf("| %-42s | %.6e | %-4s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
5208
5209 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5210 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5211 }
5212
5213
5214 static void test_sgemm(int M, int N, int K) {
5215
5216     size_t size_a = (size_t)M * K * sizeof(float);
5217     size_t size_b = (size_t)K * N * sizeof(float);
5218     size_t size_c = (size_t)M * N * sizeof(float);
5219
5220     float *h_a = (float *)malloc(size_a);
5221     float *h_b = (float *)malloc(size_b);
5222     float *h_c_ref = (float *)malloc(size_c);
5223
5224     srand(42);
5225     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5226     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5227
5228     float *d_a, *d_b, *d_c;

```

```

5229 check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
5230 check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
5231 check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
5232
5233 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
5234 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
5235
5236 // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
5237 cublasHandle_t handle;
5238 cublasCreate(&handle);
5239 float alpha = 1.0f, beta = 0.0f;
5240 cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5241             &alpha, d_b, N, d_a, K, &beta, d_c, N);
5242 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
5243
5244 // Kernel
5245 cudaEvent_t start, stop;
5246 cudaEventCreate(&start);
5247 cudaEventCreate(&stop);
5248
5249 dim3 block(32, 32);
5250 dim3 grid((N + 31) / 32, (M + 31) / 32);
5251 sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
5252 check(cudaGetLastError(), "sgemm launch");
5253 check(cudaDeviceSynchronize(), "sgemm sync");
5254
5255 float *h_c = (float *)malloc(size_c);
5256 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
5257
5258 // Verify
5259 float max_err = 0.0f;
5260 for (int i = 0; i < M * N; i++) {
5261     float err = fabsf(h_c[i] - h_c_ref[i]);
5262     if (err > max_err) max_err = err;
5263 }
5264 printf("| %-42s | %.6e | %-4s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
5265
5266 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
5267
5268 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
5269 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
5270 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
5271 check(cudaGetLastError(), "sgemm_vec4 launch");
5272 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
5273
5274 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
5275
5276 max_err = 0.0f;
5277 for (int i = 0; i < M * N; i++) {
5278     float err = fabsf(h_c[i] - h_c_ref[i]);
5279     if (err > max_err) max_err = err;
5280 }
5281 printf("| %-42s | %.6e | %-4s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL");
5282
5283 free(h_a); free(h_b); free(h_c); free(h_c_ref);
5284 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
5285 cublasDestroy(handle);
5286 cudaEventDestroy(start);
5287 cudaEventDestroy(stop);
5288 }
5289
5290
5291 static void test_hgemm_mma(int M, int N, int K) {

```

```

5292
5293 size_t size_a = (size_t)M * K * sizeof(half);
5294 size_t size_b = (size_t)K * N * sizeof(half);
5295 size_t size_c = (size_t)M * N * sizeof(half);
5296
5297 half *h_a = (half *)malloc(size_a);
5298 half *h_b = (half *)malloc(size_b);
5299 half *h_c_ref = (half *)malloc(size_c);
5300
5301 srand(42);
5302 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5303 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5304
5305 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
5306 // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
5307 size_t size_b_t = (size_t)N * K * sizeof(half);
5308 half *h_b_t = (half *)malloc(size_b_t);
5309 for (int n = 0; n < N; n++)
5310     for (int k = 0; k < K; k++)
5311         h_b_t[n * K + k] = h_b[k * N + n];
5312
5313 half *d_a, *d_b, *d_b_t, *d_c;
5314 check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
5315 check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
5316 check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
5317 check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
5318
5319 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
5320 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
5321 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
5322
5323 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5324 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
5325 cublasHandle_t handle;
5326 cublasCreate(&handle);
5327 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5328 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5329              &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5330              &beta_h, d_c, CUDA_R_16F, N,
5331              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5332 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
5333
5334 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
5335 cudaEvent_t start, stop;
5336 cudaEventCreate(&start);
5337 cudaEventCreate(&stop);
5338
5339 constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
5340 size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
5341 dim3 block(256);
5342 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5343 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5344 check(cudaGetLastError(), "hgemm launch");
5345 check(cudaDeviceSynchronize(), "hgemm sync");
5346
5347 half *h_c = (half *)malloc(size_c);
5348 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
5349
5350 // Verify
5351 float max_err = 0.0f;
5352 for (int i = 0; i < M * N; i++) {
5353     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5354     if (err > max_err) max_err = err;

```

```

5355 }
5356 printf("| %-42s | %.6e | %-4s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL");
5357
5358 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5359 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5360 cublasDestroy(handle);
5361 cudaEventDestroy(start);
5362 cudaEventDestroy(stop);
5363 }
5364
5365
5366 static void test_hgemm_swizzle(int M, int N, int K) {
5367     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
5368     //   + Register Double Buffering (kValTileK=2, BK=32)
5369     // TN layout: C[M×N] = A[M×K] × BT[N×K]
5370     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
5371     //   (kValTileK=2, kStages=3, BK=32)
5372     // smem: kStages × (BM×BK + BN×BK) halves
5373
5374     size_t size_a = (size_t)M * K * sizeof(half);
5375     size_t size_b = (size_t)K * N * sizeof(half);
5376     size_t size_c = (size_t)M * N * sizeof(half);
5377
5378     half *h_a = (half *)malloc(size_a);
5379     half *h_b = (half *)malloc(size_b);
5380     half *h_c_ref = (half *)malloc(size_c);
5381
5382     srand(42);
5383     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5384     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5385
5386     // Kernel expects BT [N×K] row-major layout (TN layout convention).
5387     size_t size_b_t = (size_t)N * K * sizeof(half);
5388     half *h_b_t = (half *)malloc(size_b_t);
5389     for (int n = 0; n < N; n++)
5390         for (int k = 0; k < K; k++)
5391             h_b_t[n * K + k] = h_b[k * N + n];
5392
5393     half *d_a, *d_b, *d_b_t, *d_c;
5394     check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
5395     check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
5396     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
5397     check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
5398
5399     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
5400     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
5401     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
5402
5403     // cuBLAS FP16 reference
5404     cublasHandle_t handle;
5405     cublasCreate(&handle);
5406     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5407     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5408                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5409                 &beta_h, d_c, CUDA_R_16F, N,
5410                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5411     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
5412
5413     // MMA swizzle kernel (default params: kStages=3, kValTileK=2, BK=32)
5414     constexpr int BM = 128, BN = 128, BK = 32, K_STAGE_S = 3;
5415     size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
5416     dim3 block(256);
5417     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);

```



```

5418 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5419 check(cudaGetLastError(), "hgemm_swizzle launch");
5420 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
5421
5422 half *h_c = (half *)malloc(size_c);
5423 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
5424
5425 // Verify
5426 float max_err = 0.0f;
5427 for (int i = 0; i < M * N; i++) {
5428     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5429     if (err > max_err) max_err = err;
5430 }
5431 printf("| %-42s | %.6e | %-4s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "FAIL");
5432
5433 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5434 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5435 cublasDestroy(handle);
5436 }
5437
5438
5439 #if defined(NOTES_V2_ENABLE_CUTE)
5440 static void test_hgemm_cute(int M, int N, int K) {
5441     // HGEMM CuTe — SM80_16x8x16_F16F16F16_TN + Swizzle<3,3> + kStage=2
5442     // TN layout: C[MxN] = A[MxK] × B^T[NxK]
5443     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
5444     // Tile: BM=128, BN=256, BK=32, 128 threads/block
5445
5446     size_t size_a = (size_t)M * K * sizeof(half);
5447     size_t size_b = (size_t)K * N * sizeof(half);
5448     size_t size_c = (size_t)M * N * sizeof(half);
5449
5450     half *h_a = (half *)malloc(size_a);
5451     half *h_b = (half *)malloc(size_b);
5452     half *h_c_ref = (half *)malloc(size_c);
5453
5454     srand(42);
5455     for (int i = 0; i < M * K; i++)
5456         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5457     for (int i = 0; i < K * N; i++)
5458         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5459
5460     // CuTe kernel expects B^T [NxK] row-major (TN layout).
5461     size_t size_b_t = (size_t)N * K * sizeof(half);
5462     half *h_b_t = (half *)malloc(size_b_t);
5463     for (int n = 0; n < N; n++)
5464         for (int k = 0; k < K; k++)
5465             h_b_t[n * K + k] = h_b[k * N + n];
5466
5467     half *d_a, *d_b, *d_b_t, *d_c;
5468     check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
5469     check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
5470     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
5471     check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
5472
5473     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
5474     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
5475     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
5476
5477     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5478     cublasHandle_t handle;
5479     cublasCreate(&handle);
5480     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);

```

```

5481 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5482             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5483             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5484 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
5485
5486 // CuTe kernel (Stages=2, TN layout)
5487 launch_hgemm_mma_stages_tn_cute<half, 2>(d_a, d_b_t, d_c, M, N, K);
5488 check(cudaGetLastError(), "hgemm_cute launch");
5489 check(cudaDeviceSynchronize(), "hgemm_cute sync");
5490
5491 half *h_c = (half *)malloc(size_c);
5492 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
5493
5494 // Verify
5495 float max_err = 0.0f;
5496 for (int i = 0; i < M * N; i++) {
5497     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5498     if (err > max_err) max_err = err;
5499 }
5500 printf("| %-42s | %.6e | %-4s |\n", "HGEMM CuTe Swizzle + Reg2x",
5501       max_err, max_err < 1.0f ? "PASS" : "FAIL");
5502
5503 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5504 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5505 cublasDestroy(handle);
5506 }
5507 #endif /* NOTES_V2_ENABLE_CUTE */
5508
5509
5510 #if defined(NOTES_V2_ENABLE_WGMMA)
5511 static void test_hgemm_wgmma(int M, int N, int K) {
5512     // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
5513     // TN layout: C[M×N] = A[M×K] × BT[N×K]
5514     // Kernel: hgemm_wgmma_stages_tn with default template params
5515
5516     constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;
5517
5518     // M, K must be divisible by tile dims
5519     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5520         printf("| %-42s | %-12s | %-4s |\n",
5521             "HGEMM WGMMA", "SKIP", "SKIP");
5522         return;
5523     }
5524
5525     size_t size_a = (size_t)M * K * sizeof(half);
5526     size_t size_b = (size_t)K * N * sizeof(half);
5527     size_t size_c = (size_t)M * N * sizeof(half);
5528
5529     half *h_a = (half *)malloc(size_a);
5530     half *h_b = (half *)malloc(size_b);
5531     half *h_c_ref = (half *)malloc(size_c);
5532
5533     srand(42);
5534     for (int i = 0; i < M * K; i++)
5535         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5536     for (int i = 0; i < K * N; i++)
5537         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5538
5539     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
5540     size_t size_b_t = (size_t)N * K * sizeof(half);
5541     half *h_b_t = (half *)malloc(size_b_t);
5542     for (int n = 0; n < N; n++)
5543         for (int k = 0; k < K; k++)

```

```

5544     h_b_t[n * K + k] = h_b[k * N + n];
5545
5546     half *d_a, *d_b, *d_b_t, *d_c;
5547     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5548     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5549     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5550     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5551
5552     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5553     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5554           "wgmma H2D B (cuBLAS)");
5555     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5556           "wgmma H2D B_t (kernel)");
5557
5558     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
5559     cublasHandle_t handle;
5560     cublasCreate(&handle);
5561     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5562     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5563                 CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5564                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5565     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5566           "wgmma D2H ref");
5567
5568     // Create TMA tensor maps for A and B^T
5569     // A[MxK] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5570     // B^T[NxK] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
5571     CUsensorMap *tma_a =
5572         allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5573     CUsensorMap *tma_b =
5574         allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5575
5576     // Launch WGMMMA kernel
5577     // kBlockSwizzle=false → 2D grid, no swizzle
5578     size_t smem_bytes =
5579         kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
5580     cudaFuncSetAttribute(
5581         hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
5582         false>,
5583         cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5584
5585     dim3 block(kNumThreads);
5586     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5587     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
5588         <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5589     check(cudaGetLastError(), "wgmma launch");
5590     check(cudaDeviceSynchronize(), "wgmma sync");
5591
5592     half *h_c = (half *)malloc(size_c);
5593     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5594
5595     // Verify
5596     float max_err = 0.0f;
5597     for (int i = 0; i < M * N; i++) {
5598         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5599         if (err > max_err)
5600             max_err = err;
5601     }
5602     printf("| %-42s | %.6e | %-4s |\n", "HGEMM TMA WGMMMA WS (3-stage)", max_err,
5603           max_err < 1.0f ? "PASS" : "FAIL");
5604
5605     free(h_a);
5606     free(h_b);

```

```

5607     free(h_b_t);
5608     free(h_c);
5609     free(h_c_ref);
5610     cudaFree(d_a);
5611     cudaFree(d_b);
5612     cudaFree(d_b_t);
5613     cudaFree(d_c);
5614     cudaFree(tma_a);
5615     cudaFree(tma_b);
5616     cublasDestroy(handle);
5617 }
5618 #endif /* NOTES_V2_ENABLE_WGMMA */
5619
5620 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
5621 template <int kStages, int kBlockSwizzle = 0>
5622 static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,
5623                                     half *d_b_t, half *d_c,
5624                                     CUTensorMap *tma_a, CUTensorMap *tma_b) {
5625     constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
5626     constexpr int kMmaTileM = 2, kMmaTileN = 2;
5627     constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
5628     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
5629     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
5630     constexpr int BK = kMmaK * kValTileK;
5631     constexpr int kNumThreads = 256;
5632     constexpr size_t payload_bytes =
5633         kStages * (BM * BK + BN * BK) * sizeof(half);
5634     constexpr size_t smem_bytes = payload_bytes;
5635     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
5636                             const CUTensorMap *);
5637     Kernel kernel = hgemm_tma_mma_ws_tn<
5638         kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
5639         kValTileK, kStages, kNumThreads, kBlockSwizzle>;
5640
5641     int device = 0;
5642     int max_smem = 0;
5643     cudaFuncAttributes attributes{};
5644     check(cudaGetDevice(&device), "tma_mma_ws get device");
5645     check(cudaDeviceGetAttribute(&max_smem,
5646                                 cudaDevAttrMaxSharedMemoryPerBlockOptin,
5647                                 device),
5648           "tma_mma_ws max shared memory");
5649     check(cudaFuncGetAttributes(&attributes, kernel),
5650           "tma_mma_ws function attributes");
5651     if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
5652         printf("| %-42s | %-12s | %-4s |\n",
5653               kStages == 2 ? "HGEMM TMA MMA WS (2-stage)"
5654                             : "HGEMM TMA MMA WS (3-stage)",
5655               "SMEM SKIP", "SKIP");
5656         return false;
5657     }
5658
5659     check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
5660                               smem_bytes),
5661           "tma_mma_ws set dynamic shared memory");
5662     dim3 block(kNumThreads);
5663     constexpr int kSwizzleN = 16;
5664     const int n_tiles = N / BN;
5665     const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
5666     const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
5667     dim3 grid(grid_x, M / BM, grid_z);
5668     kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5669     check(cudaGetLastError(), "tma_mma_ws launch");

```

```

5670     check(cudaDeviceSynchronize(), "tma_mma_ws sync");
5671     return true;
5672 }
5673
5674 static void test_hgemm_tma_mma_ws(int M, int N, int K) {
5675     constexpr int BM = 128, BN = 128, BK = 64;
5676     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5677         printf("| %-42s | %-12s | %-4s |\n", "HGEMM TMA MMA WS", "SKIP",
5678             "SKIP");
5679         return;
5680     }
5681
5682     const size_t size_a = size_t(M) * K * sizeof(half);
5683     const size_t size_b = size_t(K) * N * sizeof(half);
5684     const size_t size_b_t = size_t(N) * K * sizeof(half);
5685     const size_t size_c = size_t(M) * N * sizeof(half);
5686     half *h_a = static_cast<half *>(malloc(size_a));
5687     half *h_b = static_cast<half *>(malloc(size_b));
5688     half *h_b_t = static_cast<half *>(malloc(size_b_t));
5689     half *h_c = static_cast<half *>(malloc(size_c));
5690     half *h_c_ref = static_cast<half *>(malloc(size_c));
5691     srand(42);
5692     for (int i = 0; i < M * K; ++i)
5693         h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5694     for (int i = 0; i < K * N; ++i)
5695         h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5696     for (int n = 0; n < N; ++n)
5697         for (int k = 0; k < K; ++k)
5698             h_b_t[n * K + k] = h_b[k * N + n];
5699
5700     half *d_a, *d_b, *d_b_t, *d_c;
5701     check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
5702     check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
5703     check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
5704     check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
5705     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");
5706     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
5707     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5708         "tma_mma_ws H2D B_t");
5709
5710     cublasHandle_t handle;
5711     cublasCreate(&handle);
5712     half alpha = __float2half(1.0f), beta = __float2half(0.0f);
5713     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
5714         CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
5715         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5716     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5717         "tma_mma_ws D2H reference");
5718
5719     CUsensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5720     CUsensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5721     for (int block_swizzle : {0, 1}) {
5722         for (int stages : {2, 3}) {
5723             const bool launched = stages == 2
5724                 ? (block_swizzle
5725                     ? launch_hgemm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
5726                         tma_a, tma_b)
5727                     : launch_hgemm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
5728                         tma_a, tma_b))
5729                 : (block_swizzle
5730                     ? launch_hgemm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
5731                         tma_a, tma_b)
5732                     : launch_hgemm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,

```

```

5733         tma_a, tma_b));
5734     if (!launched)
5735         continue;
5736     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
5737           "tma_mma_ws D2H");
5738     float max_err = 0.0f;
5739     for (int i = 0; i < M * N; ++i)
5740         max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
5741                                           __half2float(h_c_ref[i])));
5742     printf("| %-42s | %.6e | %-4s |\n",
5743           block_swizzle
5744             ? (stages == 2 ? "HGEMM TMA MMA WS (2-stage, block swizzle)"
5745                           : "HGEMM TMA MMA WS (3-stage, block swizzle)")
5746             : (stages == 2 ? "HGEMM TMA MMA WS (2-stage)"
5747                           : "HGEMM TMA MMA WS (3-stage)"),
5748           max_err, max_err < 1.0f ? "PASS" : "FAIL");
5749 }
5750 }
5751
5752 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5753 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5754 cudaFree(tma_a); cudaFree(tma_b);
5755 cublasDestroy(handle);
5756 }
5757 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
5758
5759
5760 static void test_flash_attn(int seqlen, int head_dim) {
5761     // FlashAttention-2 with split-Q, MMA m16n8k16
5762     int B = 1, H = 8;
5763
5764     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5765
5766     srand(42);
5767     half *h_q = (half *)malloc(sz);
5768     half *h_k = (half *)malloc(sz);
5769     half *h_v = (half *)malloc(sz);
5770     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5771         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5772         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5773         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5774     }
5775
5776     // CPU reference in FP32:  $O = \text{softmax}(Q @ K^T / \sqrt{d}) @ V$ 
5777     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
5778     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5779     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5780     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5781     int count = B * H * seqlen * head_dim;
5782     for (int i = 0; i < count; i++) {
5783         ref_q[i] = __half2float(h_q[i]);
5784         ref_k[i] = __half2float(h_k[i]);
5785         ref_v[i] = __half2float(h_v[i]);
5786     }
5787
5788     float scale = 1.0f / sqrtf((float)head_dim);
5789     for (int bi = 0; bi < B * H; bi++) {
5790         for (int qi = 0; qi < seqlen; qi++) {
5791             //  $S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale$ 
5792             float smax = -INFINITY;
5793             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5794             for (int kj = 0; kj < seqlen; kj++) {
5795                 float s = 0.0f;

```



```

5796     for (int d = 0; d < head_dim; d++)
5797         s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5798             ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5799     S[kj] = s * scale;
5800     if (S[kj] > smax) smax = S[kj];
5801 }
5802 // softmax
5803 double sum_exp = 0.0;
5804 for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5805 float inv_sum = 1.0f / (float)sum_exp;
5806 // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
5807 for (int d = 0; d < head_dim; d++) {
5808     double o_acc = 0.0;
5809     for (int kj = 0; kj < seqlen; kj++)
5810         o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5811             ref_v[bi * seqlen * head_dim + kj * head_dim + d];
5812     ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5813 }
5814 free(S);
5815 }
5816 }
5817
5818 half *d_q, *d_k, *d_v, *d_o;
5819 check(cudaMalloc(&d_q, sz), "fa alloc Q");
5820 check(cudaMalloc(&d_k, sz), "fa alloc K");
5821 check(cudaMalloc(&d_v, sz), "fa alloc V");
5822 check(cudaMalloc(&d_o, sz), "fa alloc O");
5823 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5824 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5825 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5826
5827 // Template params for kHeadDim=64, kStage=2
5828 constexpr int kHeadDim = 64, kStageV = 2, kPadV = 8;
5829 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
5830 constexpr int kMmaTileSeqLenQ = 4;
5831 constexpr int kMmaTileSeqLenK = 1;
5832 constexpr int kMmaTileSeqLenP = 4;
5833 constexpr int kMmaTileHeadDimV = 1;
5834 constexpr int kValTileSeqLenQ = 1;
5835 constexpr int kValTileSeqLenK = 8;
5836 constexpr int kValTileSeqLenP = 1;
5837 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5838
5839 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5840 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
5841 size_t smem_bytes = (Br * (kHeadDim + kPadV) +
5842                     kStageV * Bc * (kHeadDim + kPadV) +
5843                     Bc * (kHeadDim + kPadV)) * sizeof(half);
5844
5845 dim3 block(128);
5846 dim3 grid((seqlen + Br - 1) / Br, B * H);
5847
5848 flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK,
5849     kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP, kMmaTileHeadDimV,
5850     kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP, kValTileHeadDimV,
5851     kStageV, kPadV>
5852     <<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
5853 check(cudaGetLastError(), "fa launch");
5854 check(cudaDeviceSynchronize(), "fa sync");
5855
5856 half *h_o = (half *)malloc(sz);
5857 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
5858

```

```

5859 float max_err = 0.0f;
5860 for (int i = 0; i < count; i++) {
5861     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
5862     if (err > max_err) max_err = err;
5863 }
5864 printf("| %-42s | %.6e | %-4s |\n", "FlashAttn-SplitQ", max_err, max_err < 1e-1f ? "PASS" : "FAIL");
5865
5866 free(h_q); free(h_k); free(h_v); free(h_o);
5867 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
5868 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
5869 }
5870
5871
5872 int main(int argc, char *argv[]) {
5873     #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
5874         cuInit(0); // Driver API init required for cuTensorMapEncodeTiled (TMA, sm_90a+)
5875     #endif
5876     #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
5877         if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
5878             int M = 128, N = 128, K = 64;
5879             if (argc > 4) {
5880                 M = atoi(argv[2]);
5881                 N = atoi(argv[3]);
5882                 K = atoi(argv[4]);
5883             }
5884             printf("=== SM120 TMA MMA WS validation ===\n");
5885             printf("| %-42s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
5886             printf("|-----|-----|-----|\n");
5887             test_hgemm_tma_mma_ws(M, N, K);
5888             return 0;
5889         }
5890     #endif
5891     int M = 1024, N = 1024, K = 1024;
5892     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
5893
5894     printf("=== notes-v2.cu verification harness ===\n");
5895     printf("| %-42s | %-12s | %-4s |\n", "Kernel", "Max Err", "Pass");
5896     printf("|-----|-----|-----|\n");
5897
5898     test_block_reduce(N);
5899     test_dot(N);
5900     test_relu(1024);
5901     test_elementwise(1024);
5902     test_histogram(1024);
5903     test_merge_attn_states(512, 16, 128);
5904     test_softmax(256);
5905     test_rms_norm(8, 128);
5906     test_layer_norm(8, 128);
5907     test_rope(8, 128);
5908     test_mat_transpose(256, 256);
5909     test_mat_transpose_padded(256, 256);
5910     test_sgemv(256, 128);
5911     test_sgemm(M, N, K);
5912     test_hgemm_mma(M, N, K);
5913     test_hgemm_swizzle(M, N, K);
5914     #if defined(NOTES_V2_ENABLE_CUTE)
5915         test_hgemm_cute(M, N, K);
5916     #endif
5917     #if defined(NOTES_V2_ENABLE_WGMMMA)
5918         test_hgemm_wgmma(M, N, K);
5919     #endif
5920     #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
5921         test_hgemm_tma_mma_ws(M, N, K);

```

```

5922 #endif
5923     test_flash_attn(1024, 64);
5924
5925     printf("=== All tests done ===\n");
5926     return 0;
5927 }
5928
5929 // =====
5930 // Quick build & run reference
5931 // =====
5932 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
5933 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
5934 //     -I ../../third-party/cutlass/include \
5935 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
5936 //
5937 // # sm_89 单独编译 + 运行:
5938 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
5939 //
5940 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
5941 //     项目内置 third-party/cutlass) :
5942 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
5943 //     -I ../../third-party/cutlass/include \
5944 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
5945 //
5946 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
5947 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5948 //     -DNOTES_V2_ENABLE_WGMMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
5949 //
5950 // # sm_90a + CuTe + WGMMMA:
5951 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
5952 //     -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMMA \
5953 //     -I ../../third-party/cutlass/include \
5954 //     -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
5955 //
5956 // # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
5957 //     RTX PRO 5000 / RTX 5090):
5958 // nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
5959 //     -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
5960 // ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024

```